

Easy Factor Graph: the flexible and efficient tool for managing undirected graphical models

Generated by Doxygen 1.9.1

1 What is EFG	1
2 Theoretical background on factor graphs	3
2.1 Preliminaries	3
2.2 Message Passing	7
2.2.1 Belief propagation	8
2.2.2 Message Passing	9
2.3 Maximum a posteriori estimation	13
2.4 Gibbs sampling	14
2.5 Sub graphs	15
2.6 Learning	15
2.6.1 Learning of unconditioned model	17
2.6.1.1 Gradient of α	17
2.6.1.2 Gradient of β	18
2.6.2 Learning of conditioned model	19
2.6.2.1 Gradient of β	20
2.6.3 Learning of modular structure	21
2.6.3.1 Gradient of α	21
2.6.3.2 Gradient of β	21
3 Samples	23
3.1 Sample 01: Potential handling	23
3.1.1 Variables creation	23
3.1.1.1 part 01	23
3.1.1.2 part 02	23
3.1.2 Factors creation	23
3.1.2.1 part 01	23
3.1.2.2 part 02	25
3.2 Sample 02: Belief propagation, part A	26
3.2.1 part 01	26
3.2.2 part 02	29
3.2.3 part 03	29
3.3 Sample 03: Belief propagation, part B	30
3.3.1 part 01	30
3.3.2 part 02	30
3.3.3 part 03	32
3.3.4 part 04	32
3.4 Sample 04: Hidden Markov model like structure	33
3.5 Sample 05: Matricial structure	33
3.6 Sample 06: Learning, part A	35
3.6.1 part 01	35
3.6.2 part 02	35
3.6.3 part 03	36

3.6.4 part 04	36
3.7 Sample 07: Learning, part B	36
3.8 Sample 08: Sub-graphing	37
3.8.1 part 01	37
3.8.2 part 02	37
4 Namespace Index	41
4.1 Namespace List	41
5 Hierarchical Index	43
5.1 Class Hierarchy	43
6 Class Index	45
6.1 Class List	45
7 Namespace Documentation	47
7.1 EFG Namespace Reference	47
7.1.1 Detailed Description	47
7.2 EFG::categoric Namespace Reference	47
7.2.1 Detailed Description	48
7.3 EFG::factor Namespace Reference	48
7.3.1 Detailed Description	49
7.3.2 Function Documentation	49
7.3.2.1 get_probabilities()	49
7.4 EFG::misc Namespace Reference	50
7.4.1 Detailed Description	50
7.4.2 Variable Documentation	51
7.4.2.1 IsGenerator	51
7.5 EFG::model Namespace Reference	51
7.5.1 Detailed Description	51
7.6 EFG::structure Namespace Reference	52
7.6.1 Detailed Description	53
7.6.2 Function Documentation	53
7.6.2.1 load_train_set()	53
8 Class Documentation	55
8.1 EFG::structure::BeliefManager Class Reference	55
8.2 EFG::structure::Structure::BinaryFactorInStructure Struct Reference	55
8.2.1 Detailed Description	55
8.3 EFG::misc::Chunk Struct Reference	56
8.4 EFG::misc::ConcurrentSafeLevels< T > Struct Template Reference	56
8.5 EFG::model::ConditionalRandomField Class Reference	56
8.5.1 Member Function Documentation	57
8.5.1.1 makeSamplesSpanningEvidences()	57

8.6 EFG::structure::Config Struct Reference	57
8.7 EFG::structure::ConfigManager Class Reference	58
8.7.1 Member Function Documentation	58
8.7.1.1 setCalibrationTolerance()	58
8.7.1.2 setMaxCalibrationIter()	58
8.7.1.3 setWorkersPoolSize()	58
8.8 EFG::structure::ScopedEvidenceUpdate::Context Struct Reference	59
8.9 EFG::Error Class Reference	59
8.10 EFG::structure::Evidence Struct Reference	60
8.10.1 Member Data Documentation	60
8.10.1.1 NOT_AN_EVIDENCE	60
8.11 EFG::structure::EvidenceRemoveManager Class Reference	60
8.11.1 Member Function Documentation	61
8.11.1.1 removeEvidences()	61
8.12 EFG::structure::EvidenceSetManager Class Reference	61
8.12.1 Member Function Documentation	62
8.12.1.1 setEvidences()	62
8.13 EFG::factor::ExponentialTrasform Struct Reference	62
8.13.1 Member Function Documentation	62
8.13.1.1 getWeight()	63
8.14 EFG::structure::MessageMetaData::FactorInfo Struct Reference	63
8.15 EFG::factor::FactorT< N, Transform > Class Template Reference	63
8.16 EFG::factor::FactorT< 1, Transform > Class Template Reference	64
8.17 EFG::misc::GeneratorMap< U, T, GenT > Struct Template Reference	64
8.18 EFG::misc::GenFromBlock< T > Class Template Reference	65
8.19 EFG::structure::GibbsSampler Class Reference	65
8.19.1 Detailed Description	65
8.19.2 Member Function Documentation	65
8.19.2.1 makeSamples()	65
8.20 EFG::model::Graph Class Reference	66
8.20.1 Detailed Description	66
8.21 EFG::categoric::GroupT< Support > Class Template Reference	67
8.22 EFG::structure::HiddenCluster Struct Reference	67
8.23 EFG::misc::Intervals Class Reference	67
8.24 EFG::misc::Intervals::IntervalsIter Struct Reference	68
8.25 EFG::misc::MemoryPool Class Reference	68
8.26 EFG::structure::MessageMetaData Struct Reference	69
8.27 EFG::structure::MessagePassingHandler Class Reference	69
8.28 EFG::model::Model< Components > Class Template Reference	70
8.29 EFG::structure::ModelBuilder Class Reference	70
8.30 EFG::structure::ModelExport Class Reference	71
8.31 EFG::structure::ModelSeed Struct Reference	71

8.32 EFG::misc::ModeToStream< M > Struct Template Reference	71
8.33 EFG::misc::ModeToStream< FileMode::READ > Struct Reference	72
8.34 EFG::misc::ModeToStream< FileMode::WRITE > Struct Reference	72
8.35 EFG::structure::Trainer::NoCB Struct Reference	72
8.36 EFG::structure::Node Struct Reference	72
8.37 EFG::factor::NullTrasform Struct Reference	73
8.38 EFG::structure::PairHasher Struct Reference	73
8.39 EFG::structure::PropagationOrder Struct Reference	73
8.40 EFG::structure::PropagationResult Struct Reference	73
8.41 EFG::structure::QueryManager Class Reference	74
8.41.1 Member Function Documentation	74
8.41.1.1 getHiddenSetMAP()	74
8.41.1.2 getMAP()	74
8.41.1.3 getMarginalDistribution()	75
8.42 EFG::misc::Range Struct Reference	75
8.43 EFG::categoric::RangeT< Support > Class Template Reference	76
8.43.1 Detailed Description	76
8.43.2 Constructor & Destructor Documentation	76
8.43.2.1 RangeT()	76
8.44 EFG::misc::Samples Class Reference	77
8.45 EFG::structure::GibbsSampler::SamplesGenerationContext Struct Reference	77
8.45.1 Member Data Documentation	77
8.45.1.1 delta_ iterations	77
8.45.1.2 transient	78
8.46 EFG::misc::Samples::SamplesIter Struct Reference	78
8.47 EFG::structure::WorkerPoolManager::ScopedActivator Struct Reference	78
8.48 EFG::structure::ScopedEvidenceUpdate Struct Reference	79
8.49 EFG::factor::SimplyAntiCorrelatedDomainGen< N > Class Template Reference	79
8.49.1 Detailed Description	79
8.50 EFG::factor::SimplyCorrelatedDomainGen< N > Class Template Reference	80
8.50.1 Detailed Description	80
8.51 EFG::misc::Slot< T > Struct Template Reference	81
8.52 EFG::structure::Structure Struct Reference	81
8.53 EFG::structure::Trainer Struct Reference	82
8.53.1 Member Function Documentation	82
8.53.1.1 train_model_with_cb()	82
8.54 EFG::structure::Tunability Struct Reference	83
8.55 EFG::structure::Tunability::TunableBinaryFactor Struct Reference	83
8.56 EFG::structure::Tunability::TunableUnaryFactor Struct Reference	83
8.57 EFG::structure::TunableWeightsManager::TunableWeightsGradient Struct Reference	84
8.58 EFG::structure::TunableWeightsManager Class Reference	84
8.59 EFG::factor::UnaryFactorsMerger Struct Reference	84

8.60 EFG::misc::UniformSampler Class Reference	85
8.61 EFG::misc::ValuesPool Class Reference	85
8.62 EFG::structure::ModelBuilder::VariableDescription Struct Reference	86
8.63 EFG::misc::VectorCache< Ts > Class Template Reference	86
8.64 EFG::misc::VectorCache< T > Class Template Reference	86
8.65 EFG::misc::WorkerPool Class Reference	86
8.66 EFG::structure::WorkerPoolActivityListener Struct Reference	87
8.67 EFG::structure::WorkerPoolManager Class Reference	87
Index	89

Chapter 1

What is EFG

Easy Factor Graph (EFG), is a simple and efficient C++ library for managing undirected graphical models. EFG allows you to build step by step a graphical model made of unary or binary potentials, i.e. factors involving one or two variables. It contains several tools for exporting and importing graphs from textual file. EFG allows you to perform all the probabilistic queries described in Chapter 2, from marginal probabilities computation to learning the tunable parameters of a graph. All the theoretical computations described in the initial Sections of this guide are already implemented inside the library. However, you are strongly encouraged to read this guide to understand how to use such functionalities.

The rest of this guide is structured as follows. Chapter 2 will introduce the main theoretical concepts about factor graphs, with the aim of explaining the capabilities of EFG. Chapter ?? will explain the format of the xml files adopted to represent factor graphs, exploited when importing or exporting the models to or from textual files. Chapter 3 will present the examples adopted for showing how EFG works. All the remaining Chapters, will describe the structure of the classes constituting EFG ¹.

¹A similar guide, but in a html format, is also available at http://www.andreacasalino.altervista.org/__EFG_doxy_guide/index.html.

Chapter 2

Theoretical background on factor graphs

This Section will provide the basic concepts about probabilistic models. Moreover, a precise notation will be introduced and used for the rest of this document.

2.1 Preliminaries

This library is intended for managing network of categorical variables. Formally, the generic categorical variable V has a discrete domain Dom :

$$Dom(V) = \{v_0, \dots, v_n\} \quad (2.1)$$

Essentially, $Dom(V)$ contains all the possible realizations of V . The above notation will be adopted for the rest of the guide: capital letters will refer to variable names, while non capital refer to their realizations. Group of categorical variables can be considered categorical variables too, having a domain that is the Cartesian product of the domains of the variables constituting the group. Suppose X is obtained as the union of variables $V_{1,2,3,4}$, i.e. $X = \bigcup_{i=1}^4 V_i$, then:

$$Dom(X) = Dom(V_1) \times Dom(V_2) \times Dom(V_3) \times Dom(V_4) \quad (2.2)$$

The generic realization x of X is a set of realizations of the variables $V_{1,2,3,4}$, i.e. $x = \{v_1, v_2, v_3, v_4\}$. Suppose $V_{1,2,3}$ have the domains reported in the tables 2.1. The union $X = \bigcup_{i=1}^3 V_i$ is a categoric variable whose domain is made by the combinations reported in table 2.2.

The entire population of variables contained in a model is a set denoted as $\mathcal{V} = \{V_1, \dots, V_m\}$. As will be exposed in the following, the probability of $\bigcup_{V_i \in \mathcal{V}} V_i$ ¹ is computed as the product of a certain number of components called factors.

Knowing the joint probability of $V_{1,\dots,m}$, the probability distribution of a subset $S \subset \{V_1, \dots, V_m\}$ can be in general (not only for graphical models) obtained through marginalization. Assume C is the complement of S :

$$C \cup S = \bigcup_{i=1}^m V_i \quad (2.3)$$

¹ Which is the joint probability distribution of all the variables in a model

$Dom(V_1)$	$Dom(V_2)$	$Dom(V_3)$
v_{10}	v_{20}	v_{30}
v_{11}	v_{21}	v_{31}
	v_{22}	

Table 2.1 Example of domains for the group of variables $V_{1,2,3}$.

$Dom(X) = Dom(V_1 \cup V_2 \cup V_3)$
$x_0 = \{v_{10}, v_{20}, v_{30}\}$
$x_1 = \{v_{10}, v_{20}, v_{31}\}$
$x_2 = \{v_{11}, v_{20}, v_{30}\}$
$x_3 = \{v_{11}, v_{20}, v_{31}\}$
$x_4 = \{v_{10}, v_{21}, v_{30}\}$
$x_5 = \{v_{10}, v_{21}, v_{31}\}$
$x_6 = \{v_{11}, v_{21}, v_{30}\}$
$x_7 = \{v_{11}, v_{21}, v_{31}\}$
$x_8 = \{v_{10}, v_{22}, v_{30}\}$
$x_9 = \{v_{10}, v_{22}, v_{31}\}$
$x_{10} = \{v_{11}, v_{22}, v_{30}\}$
$x_{11} = \{v_{11}, v_{22}, v_{31}\}$

Table 2.2 Example of domains for the group of variables $V_{1,2,3}$.

with $C \cap S = \emptyset$, then:

$$\mathbb{P}(S = s) = \sum_{\forall \hat{c} \in Dom(C)} \mathbb{P}(S = s, C = \hat{c}) \quad (2.4)$$

In the above computation, variables in C were marginalized. Indeed they were in a certain sense eliminated, since the probability of the sub set S was of interest, no matter the realizations of all the variables in C .

A factor, sometimes also called a potential, is a positive real function describing the correlation existing among a subset of variables $D^i \subset \mathcal{V}$. Suppose factor Φ_i involves $\{X, Y, Z\}$, i.e. $D^i = \{X, Y, Z\}$. Then, $\Phi_i(X, Y, Z)$ is a function defined over $Dom(D^i)$. More formally:

$$\Phi_i(D^i) = \Phi_i(X, Y, Z) : \text{DOMAIN}(X) \times \text{DOMAIN}(Y) \times \text{DOMAIN}(Z) \longrightarrow \mathbb{R}^+ \quad (2.5)$$

The aim of Φ_i is to assume 'high' values for those combinations $d^i = \{x, y, z\}$ that are probable to appear together and low values (at least a zero) for those being improbable. The entire population of factors $\{\Phi_1, \dots, \Phi_p\}$ correlating the variables of a model, is considered for computing $\mathbb{P}(V_{1,\dots,m})$, i.e. the joint probability distribution of all the variables in the model. The energy function E of a graph is defined as the product of the factors:

$$E(V_{1,\dots,m}) = \Phi_1(D^1) \cdot \dots \cdot \Phi_p(D^p) = \prod_{i=1}^p \Phi_i(D^i) \quad (2.6)$$

E is addressed for computing the joint probability distribution of the variables in $\mathcal{V}$:

$$\mathbb{P}(V_{1,\dots,m}) = \frac{E(V_{1,\dots,m})}{\mathcal{Z}} \quad (2.7)$$

where $\mathcal{Z}$ is a normalization coefficient defined as follows:

$$\mathcal{Z} = \sum_{\forall \tilde{V}_{1,\dots,m} \in Dom(\bigcup_{i=1,\dots,m} V_i)} E(\tilde{V}_{1,\dots,m}) \quad (2.8)$$

Although the general theory behind graphical models supports the existence of generic multivaried factors, this library will address only two possible types:

- Binary potentials: they involve a pair of variables.
- Unary potentials: they involve a single variable.

We can store the values in the image of a Binary potential in a two dimensional table. For instance, suppose Φ_b involves variables A and B , whose domains contains 3 and 5 possible values respectively:

$$\begin{aligned} \text{DOM}(A) &= \{a_1, a_2, a_3\} \\ \text{DOM}(B) &= \{b_1, b_2, b_3, b_4, b_5\} \end{aligned} \quad (2.9)$$

	b_0	b_1	b_2	b_3	b_4
a_0	1	4	0	0	0
a_1	0	1	0	0	0
a_2	0	0	5	0	1

Table 2.3 The values in the image of $\Phi_b(A, B)$.

a_0	a_1	a_2
0	2	0.5

Table 2.4 The values in the image of $\Phi_u(A)$.

The values assumed by $\Phi_b(A, B)$ are described by table 2.3. Essentially, $\Phi_b(A, B)$ tells us that the combinations $\{a_0, b_1\}$, $\{a_2, b_2\}$ are highly probable; while $\{a_0, b_0\}$, $\{a_1, b_1\}$ and $\{a_2, b_4\}$ are moderately probable. Let be $\Phi_u(A)$ a Unary potential involving variable A . The values characterizing Φ_u can be stored in a simple vector, see table 2.4. If $\Phi_b(A, B)$ would be the only potential in the model, the joint probability density of A and B will assume the following values ²:

$$\mathbb{P}(a_0, b_1) = \frac{\Phi_b(a_0, b_1)}{\mathcal{Z}} = \frac{4}{\mathcal{Z}} = 0.3333 \quad (2.10)$$

$$\mathbb{P}(a_2, b_2) = \frac{\Phi_b(a_2, b_2)}{\mathcal{Z}} = \frac{5}{\mathcal{Z}} = 0.4167 \quad (2.11)$$

$$\mathbb{P}(a_0, b_0) = \frac{\Phi_b(a_0, b_0)}{\mathcal{Z}} = \mathbb{P}(a_1, b_1) = \mathbb{P}(a_2, b_4) = \frac{1}{\mathcal{Z}} = 0.0833 \quad (2.12)$$

since $\mathcal{Z}$ is equal to:

$$\mathcal{Z} = \sum_{\forall i=\{0,1,2\}, \forall j=\{0,1,2,3,4\}} \Phi_b(A = a_i, B = b_j) = 12 \quad (2.13)$$

Both Unary and Binary potentials, can be of two possible classes:

- Factors. The potential is simply described by a set of values characterizing the image of the factor. $\Phi_b(A, B)$ and $\Phi_u(A)$ of the previous example are both Simple shapes. Classes `EFG::distribution::factor::cnst::Factor` and `EFG::distribution::factor::modif::Factor` handles this kind of potentials.
- Exponential Factors. They are indicated with Ψ_i and their image set is defined as follows:

$$\Psi_i(X) = \exp(w \cdot \Phi_i(X)) \quad (2.14)$$

where Φ_i is a Simple shape. Classes `EFG::distribution::factor::cnst::FactorExponential` and `EFG::distribution::factor::modif::FactorExponential` handles this kind of potentials. The weight w , can be tunable or not. In the first case, w is a free parameter whose value is decided after training the model (see Section 2.6), otherwise is a constant. Exponential factors with fixed weights will be denoted with $\bar{\Psi}_i$.

Figure 2.1 resumes all the possible categories of factors that can be present in the models handled by this library.

Figure 2.2 reports an example of undirected graph. Set $\mathcal{V}$ is made of 4 variables: A, B, C, D . There are 5 Binary potentials and 2 Unary ones. The graphical notation adopted for Fig. 2.2 will be adopted for the rest of this guide. Weights α, β, γ and δ are assumed for respectively $\Psi_{AC}, \Psi_{AB}, \Psi_{CD}, \Psi_B$. For the sake of clarity, the joint probability of the variables in Fig. 2.2 is computable as follows:

$$\begin{aligned} \mathbb{P}(A, B, C, D) &= \frac{E(A, B, C, D)}{\mathcal{Z}(\alpha, \beta, \gamma, \delta)} = \frac{E(A, B, CD)}{\sum_{\tilde{A}, \tilde{B}, \tilde{C}, \tilde{D}} E(\tilde{A}, \tilde{B}, \tilde{C}, \tilde{D})} \\ E(A, B, C, D) &= \Phi_A(A) \cdot \exp(\alpha \Phi_{AC}(A, C)) \cdot \exp(\beta \Phi_{AB}(A, B)) \cdot \dots \\ &\dots \Phi_{BC}(B, C) \cdot \exp(\gamma \Phi_{CD}(C, D)) \cdot \Phi_{BD}(B, D) \cdot \exp(\delta \Phi_B(B)) \end{aligned} \quad (2.15)$$

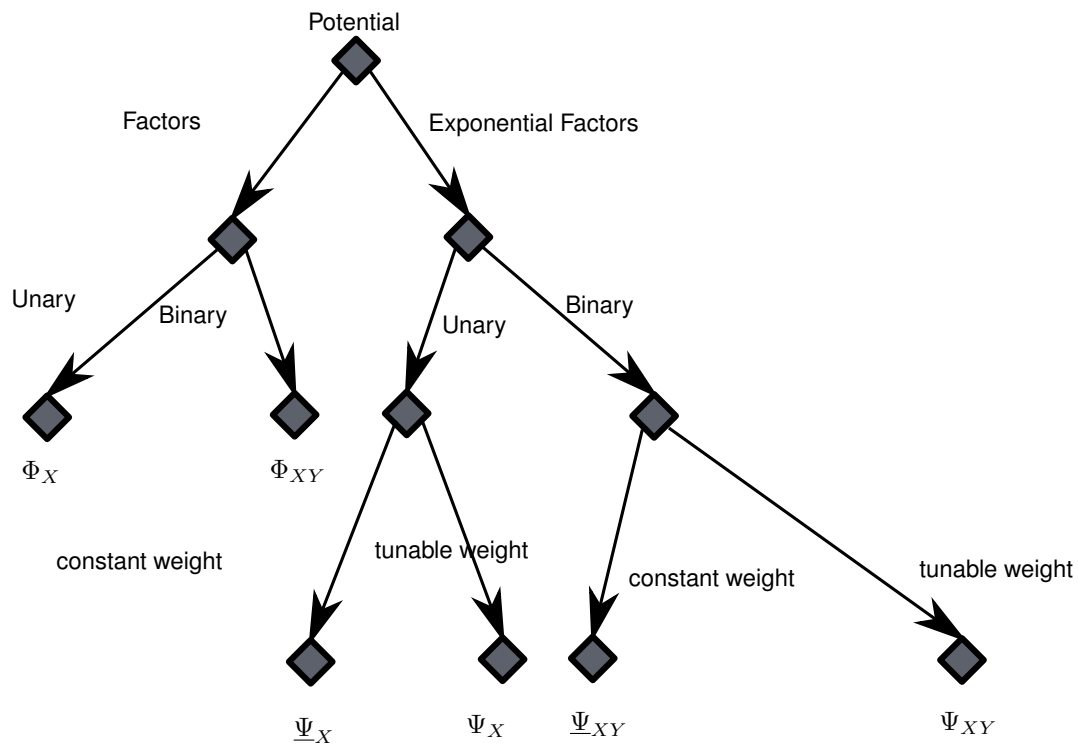


Figure 2.1 All the possible categories of factors, with the corresponding notation.

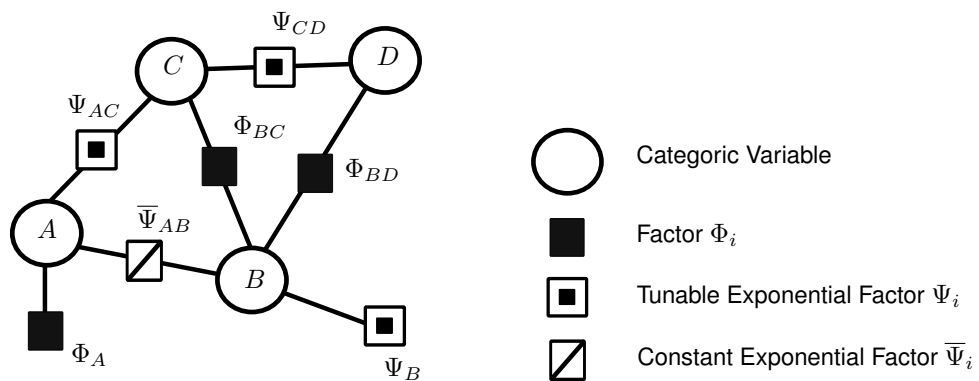


Figure 2.2 Example of graph: the legend of the right applies.

Graphical models are mainly used for performing belief propagation. Subset $\mathcal{O} = \{O_1, \dots, O_f\} \subset \mathcal{V}$ is adopted for denoting the set of evidences: those variables in the net whose value become known. $\mathcal{O}$ can be dynamical or not. The hidden variables are contained in the complementary set $\mathcal{H} = \{H_1, \dots, H_t\}$. Clearly $\mathcal{O} \cup \mathcal{H} = \mathcal{V}$ and $\mathcal{O} \cap \mathcal{H} = \emptyset$. H will be used for referring to the union of all the variables in the hidden set:

$$H = \bigcup_{i=1}^t H_i \quad (2.16)$$

while O is used for indicating the evidences:

$$O = \bigcup_{i=1}^f O_i \quad (2.17)$$

Knowing the joint probability distribution of variables in $\mathcal{V}$ (equation (2.7)) the conditional distribution of H w.r.t. O can be determined as follows:

$$\begin{aligned} \mathbb{P}(H = h | O = o) &= \frac{\mathbb{P}(H = h, O = o)}{\sum_{\hat{h} \in \text{Dom}(H)} \mathbb{P}(H = \hat{h}, O = o)} \\ &= \frac{E(h, o)}{\sum_{\hat{h} \in \text{Dom}(H)} E(\hat{h}, o)} = \frac{E(h, o)}{\mathcal{Z}(o)} \end{aligned} \quad (2.18)$$

The above computations are not actually done, since the number of combinations in the domain of $\mathcal{H}$ is huge also when considering a low-medium size graph. On the opposite, the marginal probability $\mathbb{P}(H_i = h_i | O = 0)$ of a single variable in $H_i \in \mathcal{H}$ is computationally tractable. Formally $\mathbb{P}(H_i = h_i | O = 0)$ is defined as follows:

$$\mathbb{P}(H_i = h_i | O = o) = \sum_{\tilde{h} \in \{\mathcal{H} \setminus H_i\}} \mathbb{P}(H_i = h_i, \tilde{h} | O = o) \quad (2.19)$$

The above marginal distribution is essentially the conditional distribution of H_i w.r.t. O , no matter the other variables in $\mathcal{H}$.

A generic Random Field is a graphical model for which set $\mathcal{O}$ (and consequently $\mathcal{H}$) is dynamical: the set of observations as well as the values assumed by the evidences may change during time. Random field are handled by class `RandomField`. Conditional Random Field are Random Field for which set $\mathcal{O}$ must be decided once and cannot change after. Only the values of the evidences during time may change. Class `ConditionalRandomField` is in charge of handling Conditional Random Field. Both Random Fields and Conditional Random Fields can be learnt knowing a training set, see Section 2.6. On the opposite, class `Graph` handles constant graphs: they are conceptually similar to Random Fields but learning is not possible. Indeed, all the Exponential Shape involved must be constant.

The rest of this Chapter is structured as follows. Section 2.2.2 will introduce the message passing algorithm, which is the pillar for performing belief propagation. Section 2.3 will expose the concept of maximum a posteriori estimation, useful when querying a graph, while Section 2.4 will address Gibbs sampling for producing a training set of a known model. Section 2.5 will present the concept of subgraph which is a useful way for computing the marginal probabilities of a sub group of variables in $\mathcal{H}$. Finally, 2.6 will discuss how the learning of a graphical model is done, with the aim of computing the weights of the Exponential shapes that are tunable.

2.2 Message Passing

Message passing is a powerful but conceptually simple algorithm adopted for propagating the belief across a net. Such a propagation is the starting point for performing many important operations, like computing the marginal distributions of single variables or obtaining sub graphs. Before detailing the steps involved in the message passing algorithm, let's start from an example of belief propagation. Without loss of generality we assume all the factors as simple Factors.

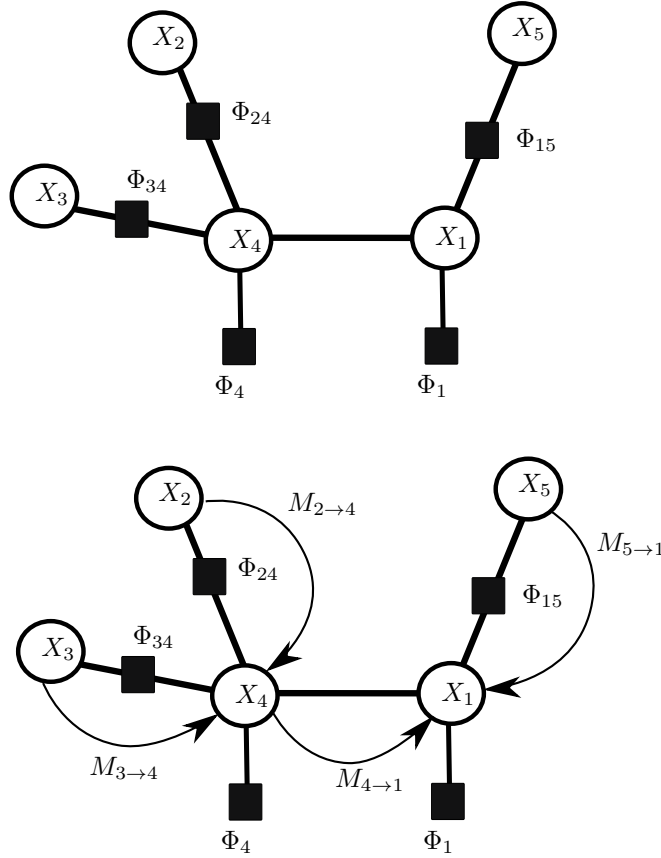


Figure 2.3 Example of graph adopted for explaining the message passing algorithm. Below are reported the messages to compute for obtaining the marginal probability of variable x_1

2.2.1 Belief propagation

Consider the graph reported in Figure 2.3. Supposing for the sake of simplicity that no evidences are available (i.e. $\mathcal{O} = \emptyset$). We are interested in computing $\mathbb{P}(X_1)$, i.e. the marginal probability of X_1 . Recalling the definition introduced in the previous Section, the marginal probability is obtained by the following computation:

$$\mathbb{P}(x_1) = \sum_{\forall \tilde{x}_{2,3,4,5} \in \cup_{i=2}^5 X_i} \mathbb{P}(x_1, \tilde{x}_{2,3,4,5}) \quad (2.20)$$

Simplifying the notation and getting rid of the normalization coefficient $\mathcal{Z}$ we can state the following:

$$\mathbb{P}(x_1) \propto \sum_{\tilde{x}_{2,3,4,5}} E(x_1, \tilde{x}_{2,3,4,5}) \quad (2.21)$$

Adopting the algebraic properties of the sums-products we can distribute the computations as follows:

$$\mathbb{P}(x_1) \propto \Phi_1(x_1) \sum_{\tilde{x}_5} \Phi_{15}(x_1, \tilde{x}_5) \sum_{\tilde{x}_4} \Phi_{14}(x_1, \tilde{x}_4) \Phi_4(\tilde{x}_4) \sum_{\tilde{x}_2} \Phi_{24}(\tilde{x}_2, \tilde{x}_4) \sum_{\tilde{x}_3} \Phi_{34}(\tilde{x}_3, \tilde{x}_4) \quad (2.22)$$

The first variable to marginalize can be $\tilde{x}_2$ or $\tilde{x}_3$, since they are involved in the last terms of the sums products. The 'messages' $M_{2 \rightarrow 4}$, $M_{3 \rightarrow 4}$ are defined as follows:

$$\begin{aligned} M_{2 \rightarrow 4}(\tilde{x}_4) &= \sum_{\tilde{x}_2} \Phi_{24}(\tilde{x}_2, \tilde{x}_4) \\ M_{3 \rightarrow 4}(\tilde{x}_4) &= \sum_{\tilde{x}_3} \Phi_{34}(\tilde{x}_3, \tilde{x}_4) \end{aligned} \quad (2.23)$$

²combinations having a null probability were omitted

Inserting $M_{2 \rightarrow 4}$ and $M_{3 \rightarrow 4}$ into equation (2.22) leads to:

$$\mathbb{P}(x_1) \propto \Phi_1(x_1) \sum_{\tilde{x}_5} \Phi_{15}(x_1, \tilde{x}_5) \sum_{\tilde{x}_4} \Phi_{14}(x_1, \tilde{x}_4) \Phi_4(\tilde{x}_4) M_{2 \rightarrow 4}(\tilde{x}_4) M_{3 \rightarrow 4}(\tilde{x}_4) \quad (2.24)$$

At this point the messages $M_{4 \rightarrow 1}$ and $M_{5 \rightarrow 1}$ can be computed in the following way:

$$\begin{aligned} M_{4 \rightarrow 1}(x_1) &= \sum_{\tilde{x}_4} \Phi_{14}(x_1, \tilde{x}_4) \Phi_4(\tilde{x}_4) M_{2 \rightarrow 4}(\tilde{x}_4) M_{3 \rightarrow 4}(\tilde{x}_4) \\ M_{5 \rightarrow 1}(x_1) &= \sum_{\tilde{x}_5} \Phi_{15}(x_1, \tilde{x}_5) \end{aligned} \quad (2.25)$$

After inserting $M_{4 \rightarrow 1}$ and $M_{5 \rightarrow 1}$ into equation (2.24) we obtain:

$$\begin{aligned} \mathbb{P}(x_1) &\propto \Phi_1(x_1) M_{4 \rightarrow 1}(x_1) M_{5 \rightarrow 1}(x_1) \\ \mathbb{P}(x_1) &= \frac{\Phi_1(x_1) M_{4 \rightarrow 1}(x_1) M_{5 \rightarrow 1}(x_1)}{\sum_{\tilde{x}_1} \Phi_1(\tilde{x}_1) M_{4 \rightarrow 1}(\tilde{x}_1) M_{5 \rightarrow 1}(\tilde{x}_1)} \end{aligned} \quad (2.26)$$

which ends the computations. Messages are, in a certain sense, able to simplify the graph sending some information from an area of the graph to another one. Indeed, variables can be replaced by messages, which can be treated as additional factors. Figure 2.3 resumes the computations exposed. Notice that the computation of $M_{4 \rightarrow 1}$ must be done after computing the messages $M_{2 \rightarrow 4}$ and $M_{3 \rightarrow 4}$, while $M_{5 \rightarrow 1}$ can be computed independently from all the others.

2.2.2 Message Passing

The aforementioned considerations can be extended to a general structured graph. Look at Figure 2.4: the computation of Message $M_{B \rightarrow A}$ can be performed only after having computed all the messages $M_{V_1, \dots, m \rightarrow B}$, i.e. the messages incoming from all the neighbours of B a part from A . Clearly $M_{B \rightarrow A}$ is computed as follows:

$$\begin{aligned} M_{B \rightarrow A}(a) &= \sum_{\tilde{b}} \Phi_{AB}(a, \tilde{b}) M_{V_1 \rightarrow B}(\tilde{b}) \cdots M_{V_m \rightarrow B}(\tilde{b}) \\ &= \sum_{\tilde{b}} \Phi_{AB}(a, \tilde{b}) \prod_{i=1}^m M_{V_i \rightarrow B}(\tilde{b}) \end{aligned} \quad (2.27)$$

Essentially, it's like having simplified the graph: we can append to A the message $M_{B \rightarrow A}(a)$ as it's a Factor, deleting factor Φ_{AB} and all the other portions of the graph, see Figure 2.4. In turn, $M_{B \rightarrow A}(a)$ will be adopted for computing the message outgoing from A .

The above elimination is not actually done: all messages incoming to all nodes of a graph are computed and stored for using them in subsequent queries. This is partially not true when considering the evidences. Indeed, when the values of the evidences are retrieved, variables in $\mathcal{O}$ are temporary deleted and replaced with messages, see Figure 2.5. Suppose variable C is connected to a variable A through a binary potential $\Phi_{AC}(A, C)$ and to variable B through Φ_{BC} . Suppose also that variable C is an evidence assuming a value equal to $\hat{c}$, then the messages sent to A and B can be computed independently as follows:

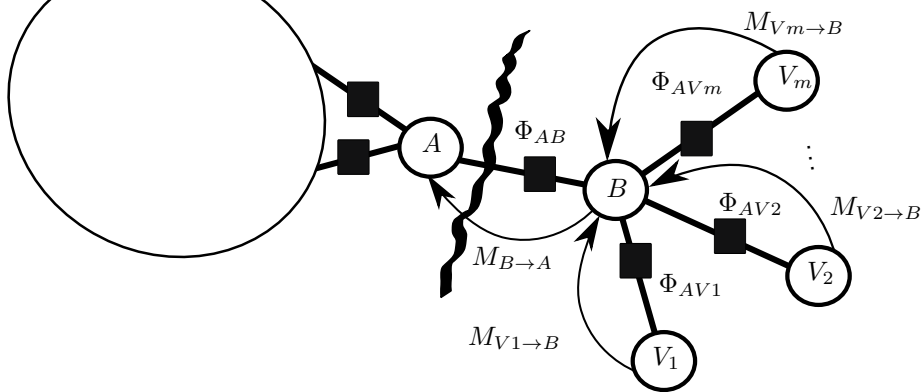
$$\begin{aligned} M_{C \rightarrow A}(a) &= \Phi_{AC}(a, \hat{c}) \\ M_{C \rightarrow B}(b) &= \Phi_{BC}(b, \hat{c}) \end{aligned} \quad (2.28)$$

Therefore all the variables that become evidences can be considered as leaves of the graph, sending messages to all the neighbouring nodes, possibly splitting an initial compact graph into many subgraphs, refer to Figure 2.5. Such computations are automatically handled by the library.

All the above considerations are valid when considering politree, i.e. graph without loops. Indeed, for these kind of graphs the message passing algorithm is able in a finite number of iterations to compute all the messages, see Figure 2.6. The same is not true when having loopy graphs (see Figure 2.7), since deadlocking situations arise: no further messages can be computed since for every nodes some incoming ones are missing. In such cases a variant of the message passing called loopy belief propagation can be adopted. Loopy belief propagation initializes all the messages to basic shapes having the values of the image all equal to 1 and then recomputes all the messages of all the variables till convergence.

You don't have to handle the latter aspect: the belief propagation mechanism is automatically handled by the library, according to the connectivity of the model for which a query is asked.

Remaining structure of the graph



Remaining structure of the graph

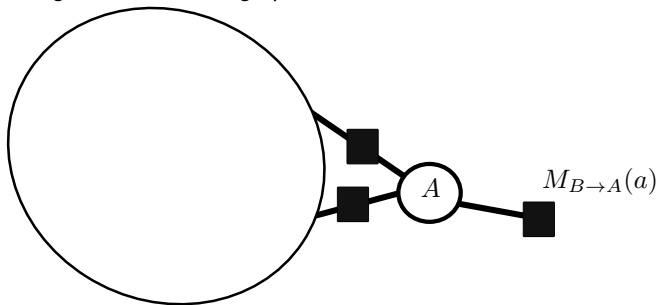


Figure 2.4 On the top the general mechanism involved in the message computation; on the bottom the simplification of the graph considering the computed message.

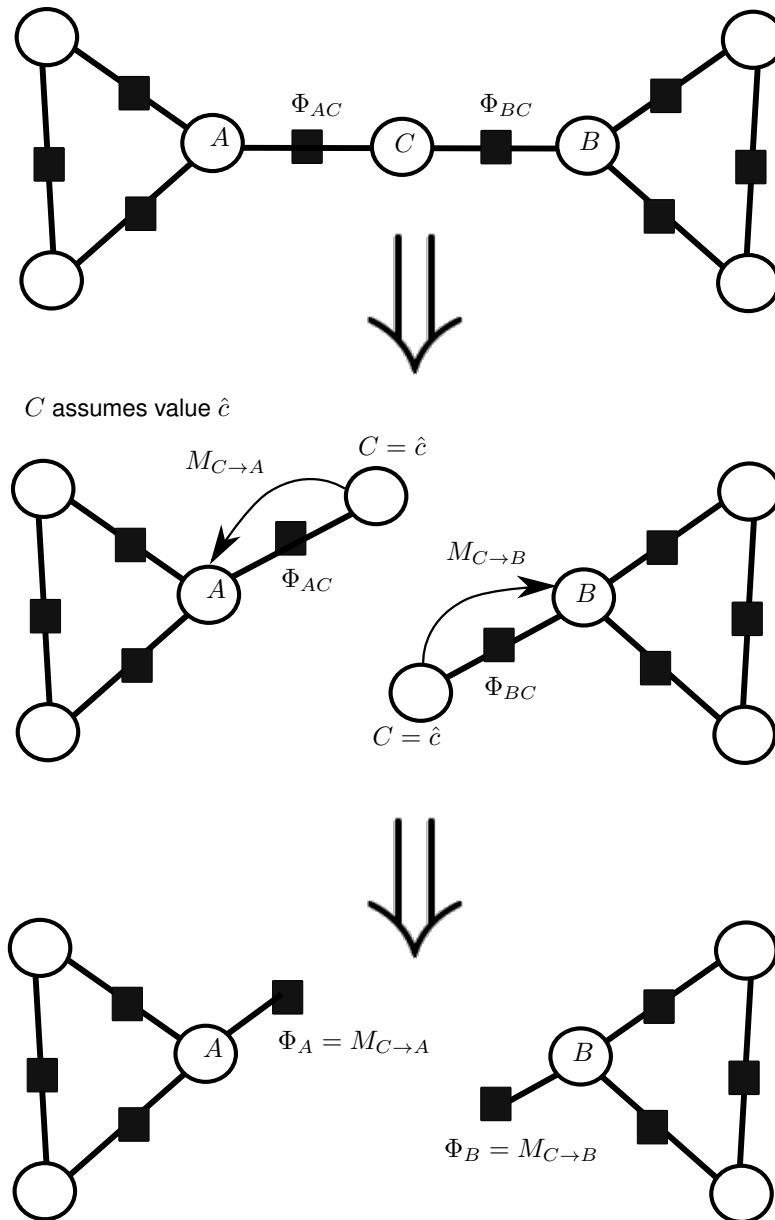


Figure 2.5 When variable C becomes an evidence, is temporary deleted from the graph, replaced by messages.

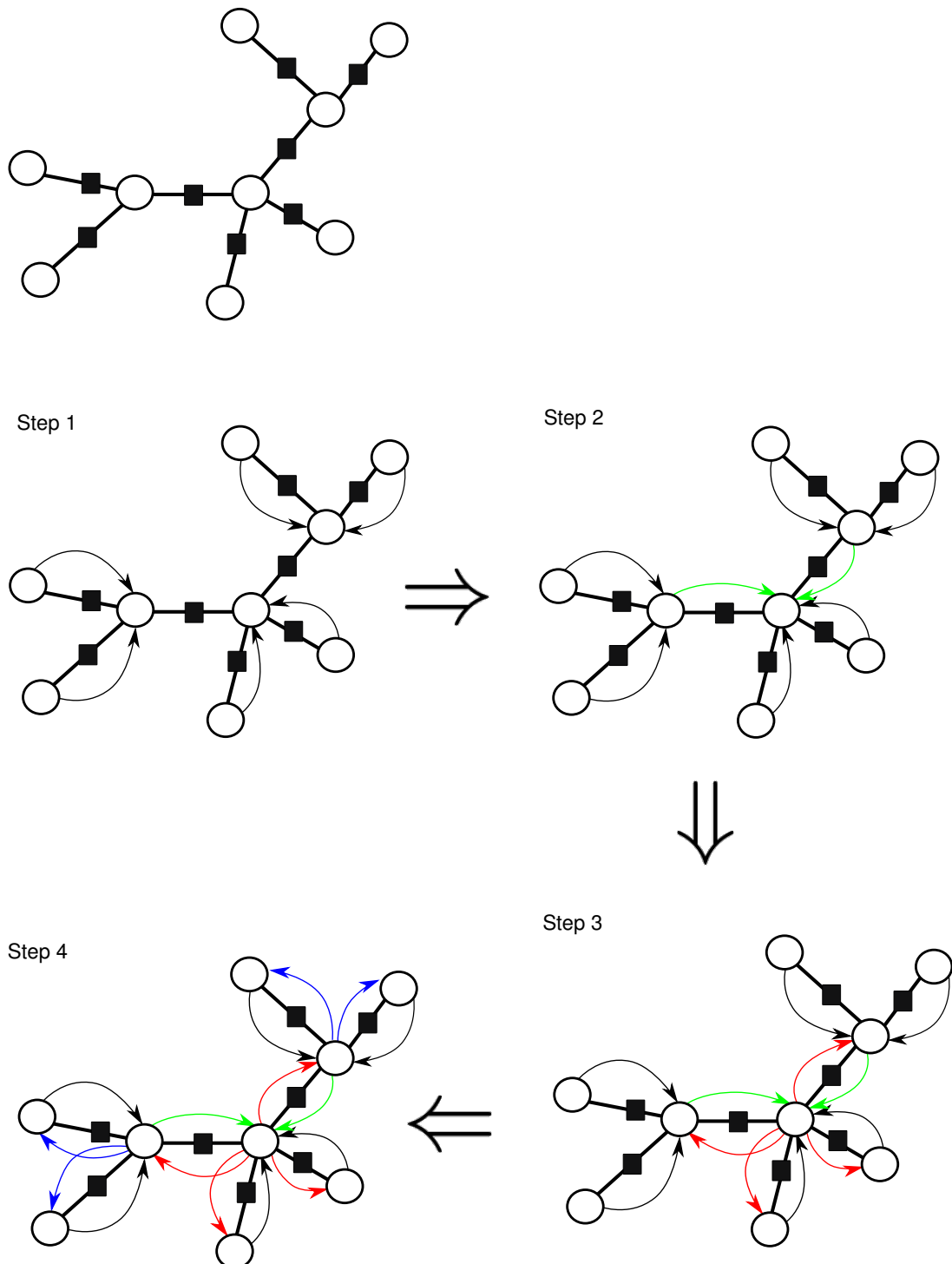


Figure 2.6 Steps involved for computing the messages of the politree represented at the top. The leaves are the first nodes for which the outgoing messages can be computed.

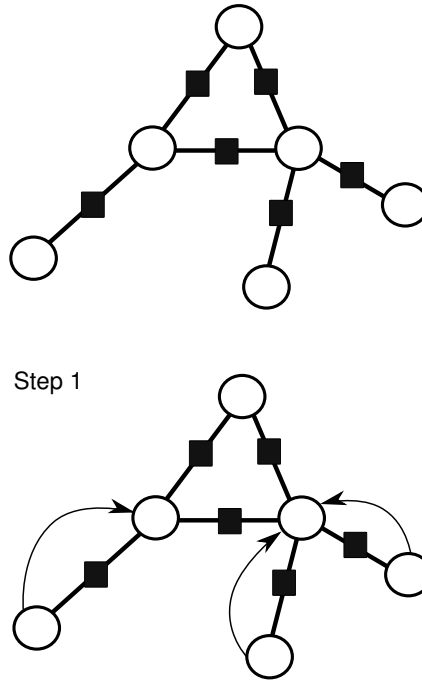


Figure 2.7 Steps involved for computing the messages on a loopy graph: after computing the messages outgoing from the leaves, a deadlock is reached since no further messages are computable.

2.3 Maximum a posteriori estimation

Suppose we are not interested in determining the marginal probability of a specific variable, but rather we want the combination in the hidden set $\mathcal{H}$ that maximises the probability $\mathbb{P}(H_{1,\dots,n}|O)$. Clearly, we could try to compute the entire distribution $\mathbb{P}(H_{1,\dots,n}|O)$ and then take the value of H maximising that distribution. However, this is not computationally possible since even for low medium size graphs the size of $Dom(\cup_{H_i \in \mathcal{H}} H_i)$ can be huge.

Maximum a posteriori estimations solve this problem: the value maximising $\mathbb{P}(H_{1,\dots,n}|O)$ is computed, without explicitly building the entire distribution $\mathbb{P}(H_{1,\dots,n}|O)$. This is achieved by performing belief propagation with a slightly different version of the message passing algorithm presented in Section 2.2.2. Referring to Figure 2.4, the message to A is computed as follows when performing a maximum a posteriori estimation:

$$M_{B \rightarrow A}(a) = \max_b \{ \Phi_{AB}(a, \tilde{b}) \prod_{i=1}^m M_{V_i \rightarrow B}(\tilde{b}) \} \quad (2.29)$$

Essentially, the summation in equation (2.27) is replaced with the max operator. After all messages are computed, the estimation $h_{MAP} = \{h_{1MAP}, h_{2MAP}, \dots\}$ is obtained by considering for every variable in $\mathcal{H}$ the value maximising:

$$h_{iMAP} = \operatorname{argmax} \{ \Phi_{H_i}(h_{iMAP}) \prod_{k=1}^L M_k(h_{iMAP}) \} \quad (2.30)$$

where $M_{1,\dots,L}$ refer to all the messages incoming to H_i . To be precise, this procedure is not guaranteed to return the value actually maximising $\mathbb{P}(H_{1,\dots,n}|O)$, but at least a strong local maximum is obtained.

At this point it is worthy to clarify that the combination $h_{MAP} = \{h_{1MAP}, h_{2MAP}, \dots\}$ could not be obtained by simply assuming for every H_i the realization maximising the marginal distribution:

$$h_{MAP} \neq \{ \operatorname{argmax}(\mathbb{P}(h_1)), \dots, \operatorname{argmax}(\mathbb{P}(h_n)) \} \quad (2.31)$$

This is due to the fact that $\mathbb{P}(H_{1,\dots,n}|O)$ is a joint probability distribution, while the marginals $\mathbb{P}(H_i)$ are not. For better understanding this aspect consider the graph reported in Figure 2.8, with the potentials Φ_{XA} , Φ_{AB} and Φ_{YB} having the images defined in table 2.5. Suppose discovering that $X = 0$ and $Y = 1$. Then, performing the standard message passing algorithm explained in the previous Section we obtain the messages reported in Figure

	b_0	b_1		x_0	x_1		y_0	y_1
a_0	2	0	a_0	1	0.1	b_0	1	0.1
a_1	0	2	a_1	0.1	1	b_1	0.1	1

Table 2.5 Factors involved in the graph of Figure 2.8.

A	B	$E(A, B, X = 0, Y = 1)$
0	0	0.2
0	1	0
1	0	0
1	1	0.2

Table 2.6 Factors involved in the graph of Figure 2.8.

2.8. Clearly individual marginals for A and B would be equal to:

$$\begin{aligned}\mathbb{P}(A) &= \begin{pmatrix} \mathbb{P}(A = 0) \\ \mathbb{P}(A = 1) \end{pmatrix} = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} \\ \mathbb{P}(B) &= \begin{pmatrix} \mathbb{P}(B = 0) \\ \mathbb{P}(B = 1) \end{pmatrix} = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix}\end{aligned}\quad (2.32)$$

Therefore, all the combinations $\{A = 0, B = 0\}$, $\{A = 0, B = 1\}$, $\{A = 1, B = 0\}$, $\{A = 1, B = 1\}$ maximise $\mathbb{P}(A, B|O)$. However, it easy to prove that $E(A, B, X, Y)$ assumes the values reported in table 2.6. Therefore, the combinations actually maximising the joint distribution $\mathbb{P}(A, B|O)$ are $\{A = 0, B = 0\}$ and $\{A = 1, B = 1\}$, leading to a different result.

2.4 Gibbs sampling

Gibbs sampling is a Monte Carlo method for obtaining samples from a joint distribution of variables $X_1, \dots, m$, without explicitly compute that distribution. Indeed, Gibbs sampling is an iterative method which requires every time to determine the conditional distribution of a single variable X_i w.r.t to all the others in the group.

More formally the algorithm starts with an initial combination of values $\{x_1, \dots, m\}$ for the variable $\cup_{i=\{1, \dots, m\}} X_i$. At every iteration, all the values of that combination are recomputed. At the j^{th} iteration the value of x_k^{j+1} for the subsequent iteration is obtaining by sampling from the following marginal distribution:

$$x_k^{j+1} \sim \mathbb{P}(x_k | x_{\{1, \dots, m\} \setminus k}^j) \quad (2.33)$$

After an initial transient, the samples cumulated during the iterations can be considered as drawn from the joint distribution involving group $X_1, \dots, m$.

This algorithm can be easily applied to graphical model. Indeed the methodologies exposed in Section 2.2.2 can be applied for determining the conditional distribution of a single variable $H_i \in \mathcal{H}$ w.r.t all the others (as well the evidences in $\mathcal{O}$), assuming all variables in $\mathcal{H} \setminus H_i$ as additional observations and computing the marginal probability of H_i .

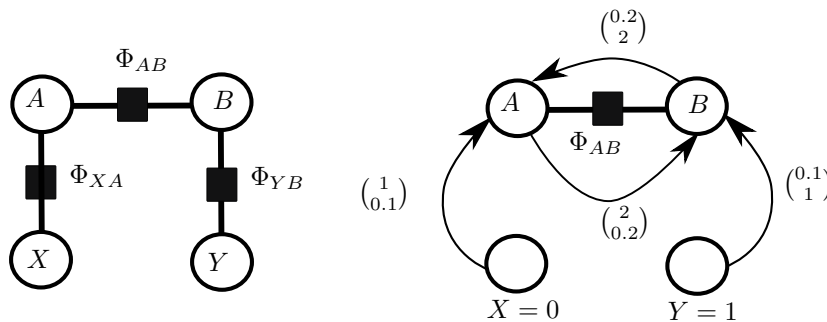


Figure 2.8 Example of graph adopted. When the evidences are retrieved, the messages computed by making use of the message passing algorithm are reported below.

2.5 Sub graphs

As explained in Section 2.2.2, the marginal probability of a variable $H_i \in \mathcal{H}$ can be efficiently computed by considering the messages produced by the message passing algorithm. The same messages can be also used for performing graph reduction, with the aim to model the joint probability distribution of a subset of variables $\{H_1, H_2, H_3\} \subset \mathcal{H}$, i.e. $\mathbb{P}(H_{1,2,3}|O)$. The latter quantity is the marginal probability of the subset of variables of interest.

The aim of message passing is essentially to simplify the graph, condensing all the belief information into the messages. Such property is exploited for computing sub graphs. Without loss of generality assume from now on $\mathcal{O} = \emptyset$. Consider the graph in Figure 2.9 and suppose we are interested in modelling $\mathbb{P}(A, B, C)$, no matter the values of the other variables. After computing all the messages exploiting message passing, the sub graph reported in Figure 2.9 is the one modelling $\mathbb{P}(A, B, C)$. Actually, that sub graph is a graphical model itself, for which all the properties exposed so far hold. For example the energy function E is computable as follows:

$$E(A = a, B = b, C = c) = \Phi_{AB}(a, b)\Phi_{BC}(b, c)\Phi_{AC}(a, c)M_{X \rightarrow A}(a)M_{Y \rightarrow B}(b) \quad (2.34)$$

while the joint probability of A, B and C can be computed in this way:

$$\mathbb{P}(A = a, B = b, C = c) = \frac{E(a, b, c)}{\sum_{\forall \tilde{a}, \tilde{b}, \tilde{c}} E(\tilde{a}, \tilde{b}, \tilde{c})} \quad (2.35)$$

Notice that in this case the graph is significantly smaller than the originating one, implying that the above computations can be performed in an acceptable time.

Also Gibbs sampling can be applied to a reduced graph, producing samples drawn from the marginal probability $\mathbb{P}(A, B, C)$.

The reduction described so far is always possible when considering a subset of variables forming a connected sub-portion of the original graph, i.e. after reduction there must be a unique sub structure. For instance, variables X and Y of the graph in Figure 2.10 do not respect the latter specification, meaning that it is not possible to build a sub graph involving X and Y .

2.6 Learning

The aim of learning is to determine the optimal values for the w (equation (2.14)) of all the tunable potentials (see Section 2.1) Ψ . To this aim two cases must be distinguished:

- Learning must be performed for a RandomField: see Section 2.6.1
- Learning must be performed for a ConditionalRandomField: see Section 2.6.2

No matter the case, the population of tunable weights, i.e. the weights of the tunable Exponential Factors of the model, will be indicated with W :

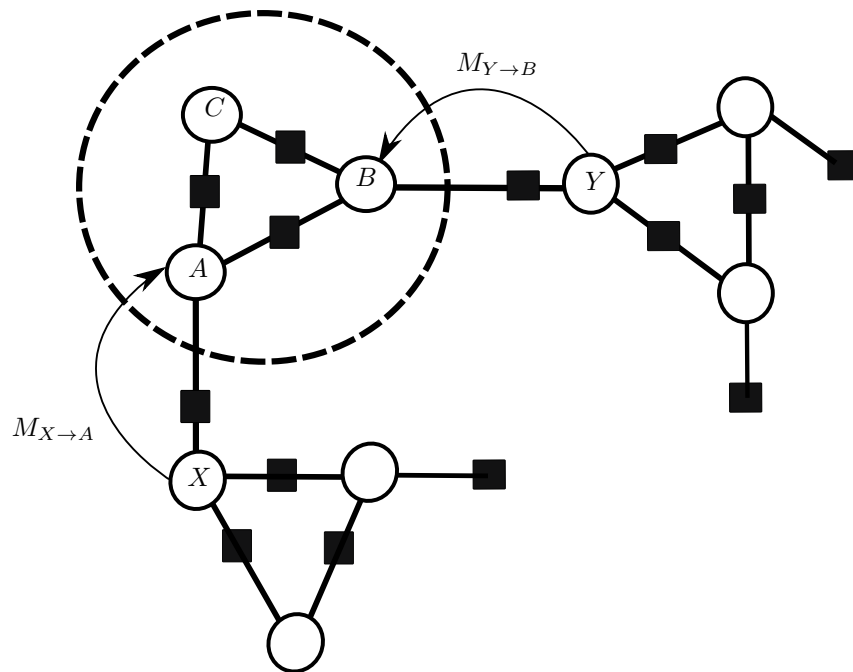
$$W = \{w_1, \dots, w_D\} \quad (2.36)$$

w_i will refer to the i^{th} free parameter of the model. For the purpose of learning, we assume $\mathcal{O} = \emptyset$. Learning considers a training set $T = \{t_1, \dots, t_N\}$ made of realizations of the joint distribution correlating all the variables in $\mathcal{V}$, no matter the fact that they are involved in tunable or non tunable potentials. As exposed in Section 2.1, if W is known, the probability of a combination t_j can be evaluated as follows:

$$\mathbb{P}(t_j) = \frac{E(t_j, W)}{\mathcal{Z}(W)} \quad (2.37)$$

At this point we can observe that the energy function is the product of two main factors: one depending from t_j and W and the other depending only upon t_j representing the contribution of all the non tunable potentials (Factors and Exponential Factors, see Section 2.1):

$$\begin{aligned} E(t_j, W) &= \exp(w_1 \Phi_1(t_j)) \cdots \exp(w_D \Phi_D(t_j)) \cdot E_0(t_j) \\ &= \exp\left(\sum_{i=1}^D w_i \Phi_i(t_j)\right) \cdot E_0(t_j) \end{aligned} \quad (2.38)$$



Sub graph involving A, B, C

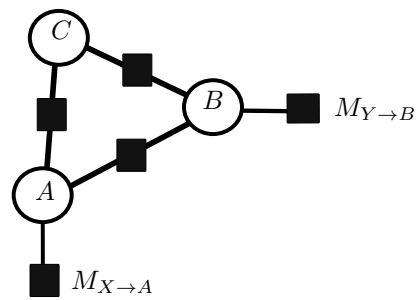


Figure 2.9 Example of graph reduction.

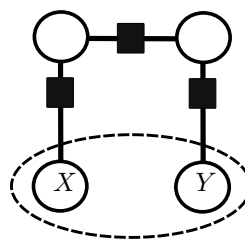


Figure 2.10 Example of a subset of variables for which the graph reduction is not possible.

The likelihood function L can be defined as follows:

$$L = \prod_{t_j \in T} \mathbb{P}(t_j) \quad (2.39)$$

passing to the logarithmic likelihood and dividing by the training set size N we obtain:

$$\begin{aligned} J = \frac{\log(L)}{N} &= \sum_{t_j \in T} \frac{\log(\mathbb{P}(t_j))}{N} \\ &= \sum_{t_j \in T} \frac{\log(E(t_j, W)) - \log(\mathcal{Z}(W))}{N} \\ &= \frac{1}{N} \sum_{t_j \in T} \log(E(t_j, W)) - \log(\mathcal{Z}(W)) \\ &= \frac{1}{N} \sum_{t_j \in T} \left(\sum_{i=1}^D w_i \Phi_i(t_j) \right) - \log(\mathcal{Z}(W)) + \dots \\ &+ \frac{1}{N} \sum_{t_j \in T} \log(E_0(t_j)) \end{aligned} \quad (2.40)$$

The aim of learning become essentially to find the value of W maximising J . This is typically done iteratively, by searching at every iteration the optimum along a direction similar to the one given by the gradient $\frac{\partial J}{\partial W}$. The way the gradient is exploited to update the searching direction characterize the kind of aproach.

The basic gradient descend approach assumes the searching direction equal to the gradient, while Quasi Newton methods (<https://www.jstor.org/stable/2006193>) use an estimation of the hessian (computed using only the gradient variation) in order to correct the direction given by the gradient. Non linear conjugate approach (<https://www.caam.rice.edu/~yzhang/caam554/pdf/cgsurvey.pdf>) implements memory-based approaches that compute the new searching direction by considering both the actual value of the gradient as well as the old used direction.

The above methods are already implemented inside this library.

2.6.1 Learning of unconditioned model

The computations to perform for evaluating the gradient $\frac{\partial J}{\partial W}$ in case of unconditioned model will be exposed in this Section. Notice that in equation (2.40), term $\sum_{t_j \in T} \log(E_0(t_j))$ is constant and consequently will be not considered for computing the gradient of J . Equation (2.40) can be rewritten as follows:

$$\begin{aligned} J &= \alpha(T, W) - \beta(W) \\ \alpha &= \frac{1}{N} \sum_{t_j \in T} \left(\sum_{i=1}^D w_i \Phi_i(t_j) \right) \end{aligned} \quad (2.41)$$

$$\beta = \log(\mathcal{Z}(W)) \quad (2.42)$$

α is influenced by T , while the same is not valid for β .

2.6.1.1 Gradient of α

By the analysis of the equation (2.41) it is clear that:

$$\frac{\partial \alpha}{\partial w_i} = \frac{1}{N} \sum_{t_j \in T} \Phi_i(t_j) \quad (2.43)$$

2.6.1.2 Gradient of β

The computation of $\frac{\partial \beta}{\partial w_i}$ requires to manipulate a little bit equation (2.42). Firstly the derivative of the logarithm must be computed:

$$\frac{\partial \beta}{\partial w_i} = \frac{1}{\mathcal{Z}} \frac{\partial \mathcal{Z}}{\partial w_i} \quad (2.44)$$

The normalizing coefficient $\mathcal{Z}$ is made of the following terms (see also equation (2.7)):

$$\mathcal{Z}(W) = \sum_{\tilde{V} \in \bigcup_{i=1}^p V_i} \left(\exp\left(\sum_{i=1}^D w_i \Phi_i(\tilde{V})\right) \cdot E_0(\tilde{V}) \right) \quad (2.45)$$

Introducing equation (2.45) into (2.44) leads to:

$$\begin{aligned} \frac{\partial \beta}{\partial w_i} &= \frac{1}{\mathcal{Z}} \frac{\partial}{\partial w_i} \left(\sum_{\tilde{V}} \exp\left(\sum_{i=1}^D w_i \Phi_i(\tilde{V})\right) E_0(\tilde{V}) \right) \\ &= \frac{1}{\mathcal{Z}} \sum_{\tilde{V}} \frac{\partial}{\partial w_i} \left(\exp\left(\sum_{i=1}^D w_i \Phi_i(\tilde{V})\right) \right) E_0(\tilde{V}) \\ &= \frac{1}{\mathcal{Z}} \sum_{\tilde{V}} \exp\left(\sum_{i=1}^D w_i \Phi_i(\tilde{V})\right) E_0(\tilde{V}) \Phi_i(\tilde{V}) \\ &= \sum_{\tilde{V}} \frac{\exp\left(\sum_{i=1}^D w_i \Phi_i(\tilde{V})\right) E_0(\tilde{V})}{\mathcal{Z}} \Phi_i(\tilde{V}) \\ &= \sum_{\tilde{V}} \frac{E(\tilde{V})}{\mathcal{Z}} \Phi_i(\tilde{V}) \\ &= \sum_{\tilde{V}} \mathbb{P}(\tilde{V}) \Phi_i(\tilde{V}) \end{aligned} \quad (2.46)$$

Last term in the above equations can be further elaborated. Assume that the variables involved in potential Φ_j are $V_{1,2}$, then:

$$\begin{aligned} \frac{\partial \beta}{\partial w_i} &= \sum_{\tilde{V}} \mathbb{P}(\tilde{V}) \Phi_i(\tilde{V}) \\ &= \sum_{\tilde{V}_{1,2}} \Phi_i(\tilde{V}_{1,2}) \sum_{\tilde{V}_{3,4,\dots}} \mathbb{P}(\tilde{V}_{1,2,3,4,\dots}) \\ &= \sum_{\tilde{V}_{1,2}} \Phi_i(\tilde{V}_{1,2}) \mathbb{P}(\tilde{V}_{1,2}) \end{aligned} \quad (2.47)$$

where $\mathbb{P}(\tilde{V}_{1,2})$ is the marginal probability (see the initial part of Section 2.1) of the variables involved in the potential Φ_i , which can be easily computable by considering the sub graph containing only V_1 and V_2 as variables (see Section 2.5). Notice that in case Φ_i is a unary potential the same holds, considering the marginal distribution of the single variable involved by Φ_i :

$$\frac{\partial \beta}{\partial w_i} = \sum_{\forall \tilde{V}_1} \Phi_i(\tilde{V}_1) \mathbb{P}(\tilde{V}_1) \quad (2.48)$$

which can be easily obtained through the message passing algorithm (Section 2.2.2).

After all the manipulations performed, the gradient $\frac{\partial J}{\partial w_i}$ has the following compact expression:

$$\frac{\partial J}{\partial w_i} = \frac{1}{N} \sum_{j=1}^N \Phi_i(D_j^i) - \sum_{\tilde{D}^i} \mathbb{P}(\tilde{D}^i) \Phi_i(\tilde{D}^i) \quad (2.49)$$

2.6.2 Learning of conditioned model

For such models learning is more demanding as will be exposed. Recalling the definition provided in the final part of Section 2.1, Conditional Random Fields are graphs for which the set of observations $\mathcal{O}$ is fixed. The training set T is made of realizations of both $\mathcal{H}$ and $\mathcal{O}$:

$$\begin{aligned} T &= \{t_1, \dots, t_N\} \\ &= \{\{h_1, o_1\}, \dots, \{h_N, o_N\}\} \end{aligned} \quad (2.50)$$

We recall, equation (2.18), that the conditional probability of the hidden variables w.r.t. the observed ones is defined as follows:

$$\begin{aligned} \mathbb{P}(h_j, o_j) &= \frac{E(h_j, o_j, W)}{\mathcal{Z}(o_j, W)} \\ E(h_j, o_j, W) &= \exp\left(\sum_{i=1}^D w_i \Phi_i(h_j, o_j)\right) E_0(h_j, o_j) \\ \mathcal{Z}(o_j, W) &= \sum_{\tilde{h}} E(\tilde{h}, o_j, W) \end{aligned} \quad (2.51)$$

The aim of learning is to maximise a likelihood uncton L defined in this case as follows:

$$L = \prod_{h_j \in T} \mathbb{P}(h_j | o_j) \quad (2.52)$$

Passing to the logarithms and dividing by the training set size we obtain the following objective function J :

$$\begin{aligned} J &= \frac{\log(L)}{N} \\ &= \frac{1}{N} \sum_{h_j, o_j \in T} \log(E(h_j, o_j, W)) - \frac{1}{N} \sum_{h_j, o_j \in T} \log(\mathcal{Z}(o_j, W)) \\ &= \frac{1}{N} \sum_{h_j, o_j \in T} \left(\sum_{i=1}^D w_i \Phi_i(h_j, o_j) \right) - \frac{1}{N} \sum_{h_j, o_j \in T} \log(\mathcal{Z}(o_j, W)) \\ &\quad + \frac{1}{N} \sum_{h_j, o_j \in T} \log(E_0(h_j, o_j)) \end{aligned} \quad (2.53)$$

Neglecting E_0 which not depends upon W , equation (2.53) can be rewritten as follows:

$$\begin{aligned} J &= \alpha(T, W) - \beta(T, W) \\ \alpha(T, W) &= \frac{1}{N} \sum_{h_j, o_j} \left(\sum_{i=1}^D w_i \Phi_i(h_j, o_j) \right) \\ \beta(T, W) &= \frac{1}{N} \sum_{o_j} \log(\mathcal{Z}(o_j, W)) \end{aligned} \quad (2.54)$$

At this point, an important remark must be done: differently from the β defined in equation (2.42), $\beta(T, W)$ of conditioned model is a function of the training set. The latter observation has an important consequence: when performing learning of unconditioned model, belief propagation (i.e. the computation of the messages through message passing with the aim of computing the marginal probabilities of the groups of variables involved in the factor of the model) must be performed once for every iteration of the gradient descend; on the opposite when considering conditioned model, belief propagation must be performed at every iteration for every element of the training set, see equation (2.58). This makes the learning of conditioned models much more computationally demanding. This price is paid in order to not model the correlation among the observations³, which can be interesting for many applications. The computation of $\frac{\partial \alpha}{\partial w_i}$ is analogous to the one of non conditioned model, equation (2.43).

³that can be highly correlated

2.6.2.1 Gradient of β

Following the same approach in Section 2.6.1.2, the gradient of β can be computed as follows:

$$\begin{aligned}
 \frac{\partial \beta}{\partial w_i} &= \frac{1}{N} \sum_{j=1}^N \frac{\partial \log(\mathcal{Z}(o_j, W))}{\partial w_i} \\
 &= \frac{1}{N} \sum_{j=1}^N \frac{1}{\mathcal{Z}(o_j)} \frac{\partial \mathcal{Z}(o_j, W)}{\partial w_i} \\
 &= \frac{1}{N} \sum_{j=1}^N \frac{\partial}{\partial w_i} \left(\sum_{\tilde{h}} \exp\left(\sum_{i=1}^D w_i \Phi_i(\tilde{h}, o_j)\right) E_0(\tilde{h}, o_j) \right) \\
 &= \frac{1}{N} \sum_{j=1}^N \sum_{\tilde{h}} \left(\exp\left(\sum_{i=1}^D w_i \Phi_i(\tilde{h}, o_j)\right) E_0(\tilde{h}, o_j) \Phi_i(\tilde{h}, o_j) \right) \\
 &= \frac{1}{N} \sum_{j=1}^N \sum_{\tilde{h}} \frac{E(\tilde{h}, o_j, W)}{\mathcal{Z}(o_j)} \Phi_i(\tilde{h}, o_j) \\
 &= \frac{1}{N} \sum_{j=1}^N \sum_{\tilde{h}} \mathbb{P}(\tilde{h}|o_j) \Phi_i(\tilde{h}, o_j)
 \end{aligned} \tag{2.55}$$

Suppose the variables involved in the factor Φ_j are $\tilde{h}_{1,2}$, then:

$$\begin{aligned}
 \frac{\partial \beta}{\partial w_i} &= \frac{1}{N} \sum_{j=1}^N \sum_{\tilde{h}} \mathbb{P}(\tilde{h}|o_j) \Phi_i(\tilde{h}, o_j) \\
 &= \frac{1}{N} \sum_{j=1}^N \sum_{\tilde{h}_{1,2}} \Phi_i(\tilde{h}_{1,2}) \sum_{\tilde{h}_{3,4,\dots}} \mathbb{P}(\tilde{h}_{1,2,3,4,\dots}|o_j) \\
 &= \frac{1}{N} \sum_{j=1}^N \sum_{\tilde{h}_{1,2}} \Phi_i(\tilde{h}_{1,2}) \mathbb{P}(\tilde{h}_{1,2}|o_j)
 \end{aligned} \tag{2.56}$$

where $\mathbb{P}(\tilde{h}_{1,2}|o_j)$ is the conditioned marginal probability of group $\tilde{h}_{1,2}$ w.r.t. the observations o_j .

Grouping all the simplifications we obtain:

$$\frac{\partial J}{\partial w_i} = \frac{1}{N} \sum_{j=1}^N \Phi_i(h_j, o_j) - \frac{1}{N} \sum_{j=1}^N \left(\sum_{\tilde{h}_{1,2}} \mathbb{P}(\tilde{h}_{1,2}|o_j) \Phi_i(\tilde{h}_{1,2}) \right) \tag{2.57}$$

Generalizing:

$$\frac{\partial J}{\partial w_i} = \frac{1}{N} \sum_{j=1}^N \Phi_i(D_j^i, o_j) - \frac{1}{N} \sum_{j=1}^N \left(\sum_{\tilde{D}^i} \mathbb{P}(\tilde{D}^i|o_j) \Phi_i(\tilde{D}^i, o_j) \right) \tag{2.58}$$

2.6.3 Learning of modular structure

Suppose to have a modular structure made of repeating units as for example the graph in Figure 2.11. Every single unit has the same population of potentials and we would like to enforce this fact when performing learning. In particular we'll have some sets of Exponential shape sharing the same weight w_1 (see Figure 2.11). Motivated by this example, we included in the library the possibility to specify that a potential must share its weight with another one. Then, learning is done consistently with the aforementioned specification.

2.6.3.1 Gradient of α

Considering the model in Figure 2.11, the α part of J (equation (2.41)) can be computed as follows:

$$\alpha = \frac{1}{N} \sum_{t_j} (w_1 \Phi_1(a_{1j}, b_{1j}) + w_1 \Phi_2(a_{2j}, b_{2j}) + w_1 \Phi_3(a_{3j}, b_{3j}) + \dots + \dots + \sum_{i=2}^D w_i \Phi_i(t_j)) \quad (2.59)$$

which leads to:

$$\frac{\partial \alpha}{\partial w_1} = \frac{1}{N} \sum_{t_j} (\Phi_1(a_{1j}, b_{1j}) + \Phi_2(a_{2j}, b_{2j}) + \Phi_3(a_{3j}, b_{3j})) \quad (2.60)$$

2.6.3.2 Gradient of β

Regarding the β part of J we can write what follows:

$$\begin{aligned} \frac{\partial \beta}{\partial w_1} &= \frac{1}{Z} \frac{\partial Z}{\partial w_1} \\ &= \frac{1}{Z} \frac{\partial}{\partial w_1} \left(\sum_{\tilde{V}} \left(\exp(w_1(\Psi_1(a_{1j}, b_{1j}) + \dots \right. \right. \\ &\quad \left. \left. + \Psi_2(a_{2j}, b_{2j}) + \Psi_3(a_{3j}, b_{3j})) + \sum_{i=2}^D w_i \Phi_i(\tilde{V})) E_0(\tilde{V})) \right) \right) \\ &= \sum_{\tilde{V}} \mathbb{P}(\tilde{V}) (\Phi_1(\tilde{a}_1, \tilde{b}_1) + \Phi_2(\tilde{a}_2, \tilde{b}_2) + \Phi_3(\tilde{a}_3, \tilde{b}_3)) \\ &= \sum_{\tilde{V}} \mathbb{P}(\tilde{V}) \Phi_1(\tilde{a}_1, \tilde{b}_1) + \sum_{\tilde{V}} \mathbb{P}(\tilde{V}) \Phi_2(\tilde{a}_2, \tilde{b}_2) + \sum_{\tilde{V}} \mathbb{P}(\tilde{V}) \Phi_3(\tilde{a}_3, \tilde{b}_3) \\ &= \sum_{\tilde{A}_1, \tilde{B}_1} \mathbb{P}(\tilde{A}_1, \tilde{B}_1) \Phi_1(\tilde{A}_1, \tilde{B}_1) + \sum_{\tilde{A}_2, \tilde{B}_2} \mathbb{P}(\tilde{A}_2, \tilde{B}_2) \Phi_2(\tilde{A}_2, \tilde{B}_2) + \dots \\ &\quad \dots + \sum_{\tilde{A}_3, \tilde{B}_3} \mathbb{P}(\tilde{A}_3, \tilde{B}_3) \Phi_3(\tilde{A}_3, \tilde{B}_3) \end{aligned} \quad (2.61)$$

Notice that the gradient $\frac{\partial J}{\partial w_1}$ is the summation of three terms: the ones that would have been obtained considering separately the three potentials in which w_1 is involved (equation (2.49)):

$$\begin{aligned} \frac{\partial J}{\partial w_1} &= \frac{1}{N} \sum_{j=1}^N \Phi_1(a_i^1, b_i^1) - \sum_{\tilde{a}^1, \tilde{b}^1} \mathbb{P}(\tilde{a}^1, \tilde{b}^1) \Phi_1(\tilde{a}^1, \tilde{b}^1) + \dots \\ &\quad + \frac{1}{N} \sum_{j=1}^N \Phi_2(a_i^2, b_i^2) - \sum_{\tilde{a}^2, \tilde{b}^2} \mathbb{P}(\tilde{a}^2, \tilde{b}^2) \Phi_2(\tilde{a}^2, \tilde{b}^2) + \dots \\ &\quad + \frac{1}{N} \sum_{j=1}^N \Phi_3(a_i^3, b_i^3) - \sum_{\tilde{a}^3, \tilde{b}^3} \mathbb{P}(\tilde{a}^3, \tilde{b}^3) \Phi_3(\tilde{a}^3, \tilde{b}^3) + \dots \end{aligned} \quad (2.62)$$

The above result has a general validity, also considering conditioned graphs.

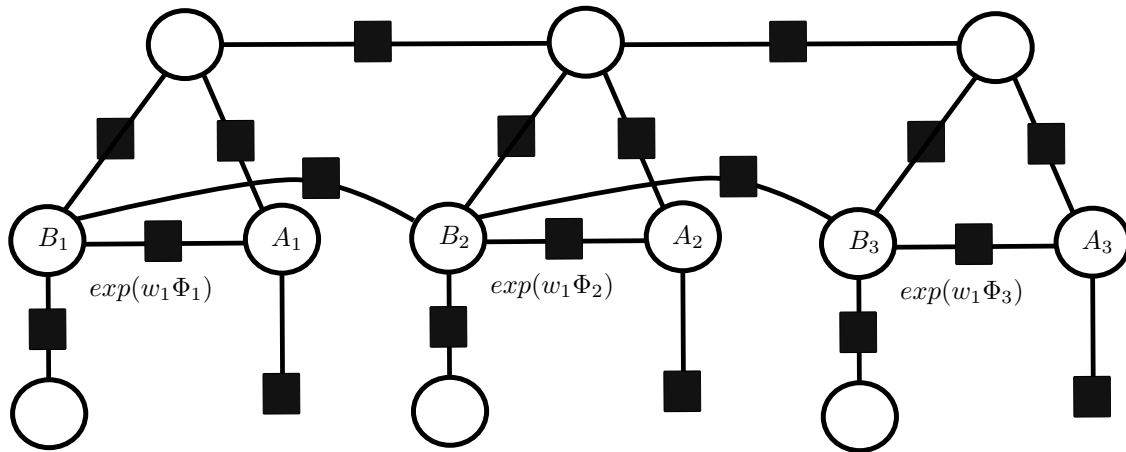


Figure 2.11 Example of modular structure: weight w_1 is simultaneously involved into potentials Φ_1, Φ_2 and Φ_3 .

Chapter 3

Samples

3.1 Sample 01: Potential handling

The aim of this series of examples is mainly to show how to handle the creation of variables and factors.

3.1.1 Variables creation

3.1.1.1 part 01

This example creates a tow initial variables A and B with a domain size equal to 2 and places them into a group storing them. Later, it adds variables C and D , with the same domain size. Finally, tries to add again C , showing the this further addition is correctly refused.

3.1.1.2 part 02

This example considers 3 variables A , B and C with a domain sizes equal to, respectively, 2,4 and 3, i.e:

$$\begin{aligned} Dom(A) &= \{a_1 = 0, a_2 = 1\} \\ Dom(B) &= \{b_1 = 0, b_2 = 1, b_3 = 2, b_4 = 3\} \\ Dom(C) &= \{c_1 = 0, c_2 = 1, c_3 = 2\} \end{aligned} \tag{3.1}$$

Then, evaluates the joint domain of: $A \cup B$, $A \cup C$ and $A \cup B \cup C$, which should results in the following combinations reported in, respectively, tables 3.1, 3.2 and 3.3.

3.1.2 Factors creation

3.1.2.1 part 01

Part 01 creates a shape factor Φ_{AB} , involving the pair of variables A and B . Both that variables have a domain size equal to 4, i.e. $Dom(A) = \{a_0 = 0, a_1 = 1, a_2 = 2, a_3 = 3\}$ and $Dom(B) = \{b_0 = 0, b_1 = 1, b_2 = 2, b_3 = 3\}$. The generic value in the image Φ_{AB} is equal to:

$$\Phi_{AB}(A = a, B = b) = a + 2 \cdot b \tag{3.2}$$

Table 3.4 reports the entire image of Φ_{AB} .

$Dom(AB) = Dom(A \cup B)$
$\{a_1, b_1\}$
$\{a_1, b_2\}$
$\{a_1, b_3\}$
$\{a_1, b_4\}$
$\{a_2, b_1\}$
$\{a_2, b_2\}$
$\{a_2, b_3\}$
$\{a_2, b_4\}$

Table 3.1 Domain of AB .

$Dom(AC) = Dom(A \cup C)$
$\{a_1, c_1\}$
$\{a_1, c_2\}$
$\{a_1, c_3\}$
$\{a_2, c_1\}$
$\{a_2, c_2\}$
$\{a_2, c_3\}$

Table 3.2 Domain of AC .

$Dom(ABC) = Dom(A \cup B \cup C)$
$\{a_1, b_1, c_1\}$
$\{a_1, b_1, c_2\}$
$\{a_1, b_1, c_3\}$
$\{a_1, b_2, c_1\}$
$\{a_1, b_2, c_2\}$
$\{a_1, b_2, c_3\}$
$\{a_1, b_3, c_1\}$
$\{a_1, b_3, c_2\}$
$\{a_1, b_3, c_3\}$
$\{a_1, b_4, c_1\}$
$\{a_1, b_4, c_2\}$
$\{a_1, b_4, c_3\}$
$\{a_2, b_1, c_1\}$
$\{a_2, b_1, c_2\}$
$\{a_2, b_1, c_3\}$
$\{a_2, b_2, c_1\}$
$\{a_2, b_2, c_2\}$
$\{a_2, b_2, c_3\}$
$\{a_2, b_3, c_1\}$
$\{a_2, b_3, c_2\}$
$\{a_2, b_3, c_3\}$
$\{a_2, b_4, c_1\}$
$\{a_2, b_4, c_2\}$
$\{a_2, b_4, c_3\}$

Table 3.3 Domain of ABC .

	$b_0 = 0$	$b_1 = 1$	$b_2 = 2$	$b_3 = 3$
$a_0 = 0$	0	2	4	6
$a_1 = 1$	1	3	5	7
$a_2 = 2$	2	4	6	8
$a_3 = 3$	3	5	7	9

Table 3.4 The values in the image of Φ_{AB} .

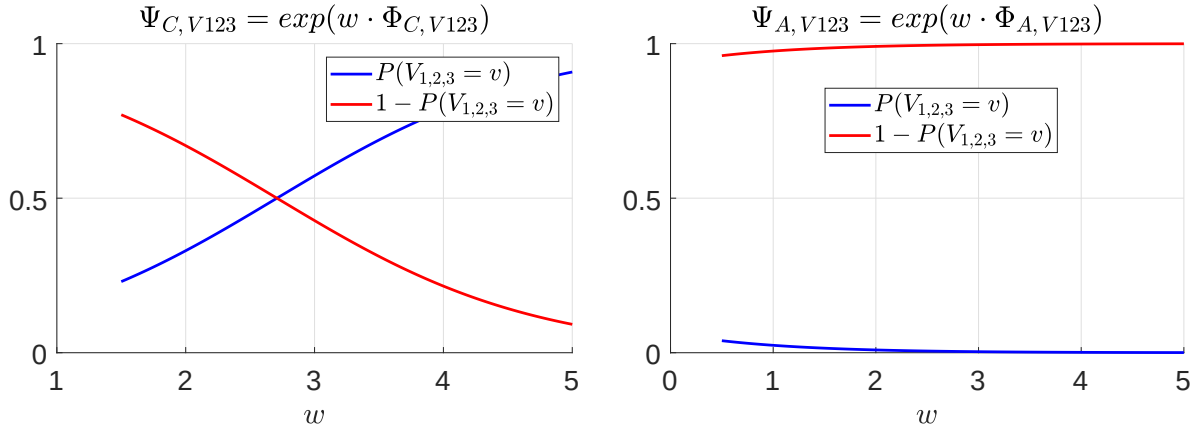


Figure 3.1 The probability $\mathbb{P}(V_1 = v, V_2 = v, V_3 = v|w)$ and its complement, when considering a ternary correlating factor, on the left, and an anti-correlating one, on the right.

3.1.2.2 part 02

Part 02 considers a ternary correlating factor $\Phi_{C, V_{123}}$, involving variables V_1 , V_2 and V_3 , each having a domain size equal to 3. Ternary factors cannot be part of a graph, but it is anyway possible to build them as distribution object. The values in the image of $\Phi_{C, V_{123}}$ are all 0, except for those combination for which V_1 , V_2 and V_3 assume the same value (0, 1 or 2) and in such cases, the image is equal to 1.

The same example builds at a second stage a ternary anti-correlating factor $\Phi_{A, V_{123}}$. The values in the image of $\Phi_{A, V_{123}}$ are all 1, except for those combination for which V_1 , V_2 and V_3 assume the same value (0, 1 or 2) and in such cases the image is equal to 0.

When considering a graph having only $\Psi_{C, V_{123}} = \exp(\Phi_{C, V_{123}} \cdot w)$ as a factor, the ripartition function Z is equal to:

$$Z = (4^3 - 4) + 4 \cdot \exp(w) \quad (3.3)$$

The probability to have as a realization a combination with the same values is equal to:

$$\mathbb{P}(V_1 = v, V_2 = v, V_3 = v) = \sum_{i=0}^3 \mathbb{P}(V_1 = i, V_2 = i, V_3 = i) \quad (3.4)$$

$$= \sum_{i=0}^3 \frac{\Psi_{C, V_{123}}(i, i, i)}{Z} \quad (3.5)$$

$$= 4 \cdot \frac{\Psi_{C, V_{123}}(0, 0, 0)}{Z} \quad (3.6)$$

$$= \frac{4 \cdot \exp(w)}{(4^3 - 4) + 4 \cdot \exp(w)} \quad (3.7)$$

The value assumed by $\mathbb{P}(V_1 = v, V_2 = v, V_3 = v|w)$ is reported in Figure 3.1, together with the complementary probability $1 - \mathbb{P}(V_1 = v, V_2 = v, V_3 = v|w)$.

When considering a graph having only $\Psi_{A, V_{123}} = \exp(\Phi_{A, V_{123}} \cdot w)$ as factor, the ripartition function Z is equal to:

$$Z = (4^3 - 4) \cdot \exp(w) + 4 \quad (3.8)$$

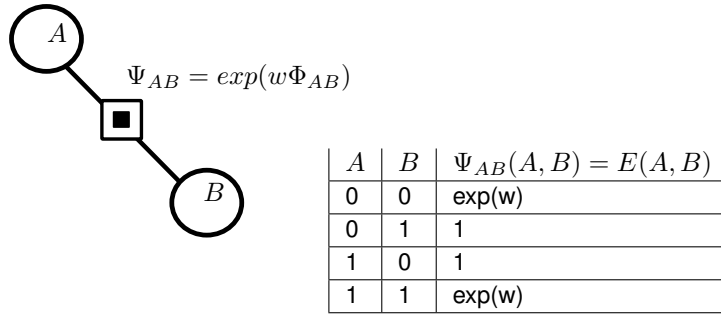


Figure 3.2 On the left the graph considered in this example, while on the right the image of factor Ψ_{AB} . Since that potential is the only one present in the graph, the values in the image of Ψ_{AB} are also the ones assume by the energy function E .

The probability to have as a realization a combination with the same values is equal to:

$$\mathbb{P}(V_1 = v, V_2 = v, V_3 = v) = \sum_{i=0}^3 \mathbb{P}(V_1 = i, V_2 = i, V_3 = i) \quad (3.9)$$

$$= \sum_{i=0}^3 \frac{\Psi_{A \ V123}(i, i, i)}{Z} \quad (3.10)$$

$$= 4 \cdot \frac{\Psi_{A \ V123}(0, 0, 0)}{Z} \quad (3.11)$$

$$= \frac{4}{(4^3 - 4) \cdot \exp(w) + 4} \quad (3.12)$$

The value assumed by $\mathbb{P}(V_1 = v, V_2 = v, V_3 = v|w)$ is reported in Figure 3.1, together with its complement $1 - \mathbb{P}(V_1 = v, V_2 = v, V_3 = v|w)$. Indeed, when the variables are correlated, i.e. they share $\Psi_{C \ V123}$, the probability $\mathbb{P}(V_1 = v, V_2 = v, V_3 = v|w)$ is big. Moreover, the more w is high (i.e. the more the variables are correlated), the more the latter probability is big. On the opposite, when the variables are anti-correlated, the opposite situation arises.

3.2 Sample 02: Belief propagation, part A

The aim of this series of examples is to show how to perform probabilistic queries on factor graphs.

3.2.1 part 01

This example creates a graph having a single binary exponential shape Ψ_{AB} , see Figure 3.2, with a A and B having a Dom size equal to 2. $\Psi_{AB} = \exp(w\Phi_{AB})$, where Φ_{AB} is a simple correlating factor. The image of Ψ_{AB} is reported in the right part of Figure 3.2.

Variable B is considered as an evidence, whose value is equal, for the first part of the example, to 0, while a value of 1 is assumed in the second part. The probability of A conditioned to B , is equal to (see equation (2.18)):

$$\mathbb{P}(A = a|B = 0) = \frac{E(A = a, B = 0)}{E(A = 0, B = 0) + E(A = 1, B = 0)} \Rightarrow \begin{bmatrix} \mathbb{P}(A = 0|B = 0) = \frac{\exp(w)}{1 + \exp(w)} \\ \mathbb{P}(A = 1|B = 0) = \frac{1}{1 + \exp(w)} \end{bmatrix} \quad (3.13)$$

$$\mathbb{P}(A = a|B = 1) = \frac{E(A = a, B = 1)}{E(A = 0, B = 1) + E(A = 1, B = 1)} \Rightarrow \begin{bmatrix} \mathbb{P}(A = 0|B = 1) = \frac{1}{1 + \exp(w)} \\ \mathbb{P}(A = 1|B = 1) = \frac{\exp(w)}{1 + \exp(w)} \end{bmatrix} \quad (3.14)$$

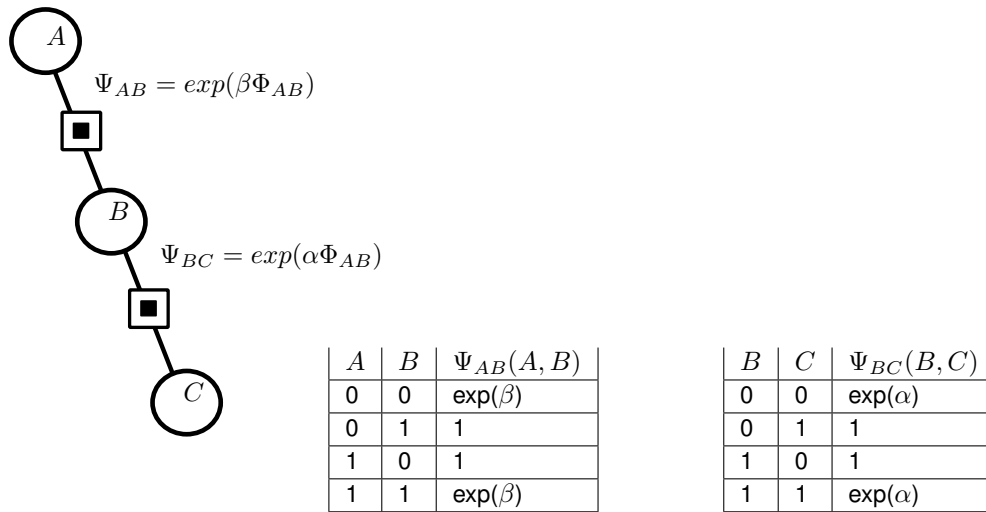


Figure 3.3 On the left the graph considered in this example, while on the right the images of factor Ψ_{AB} and Ψ_{BC} having, respectively, a weight equal to β and α .

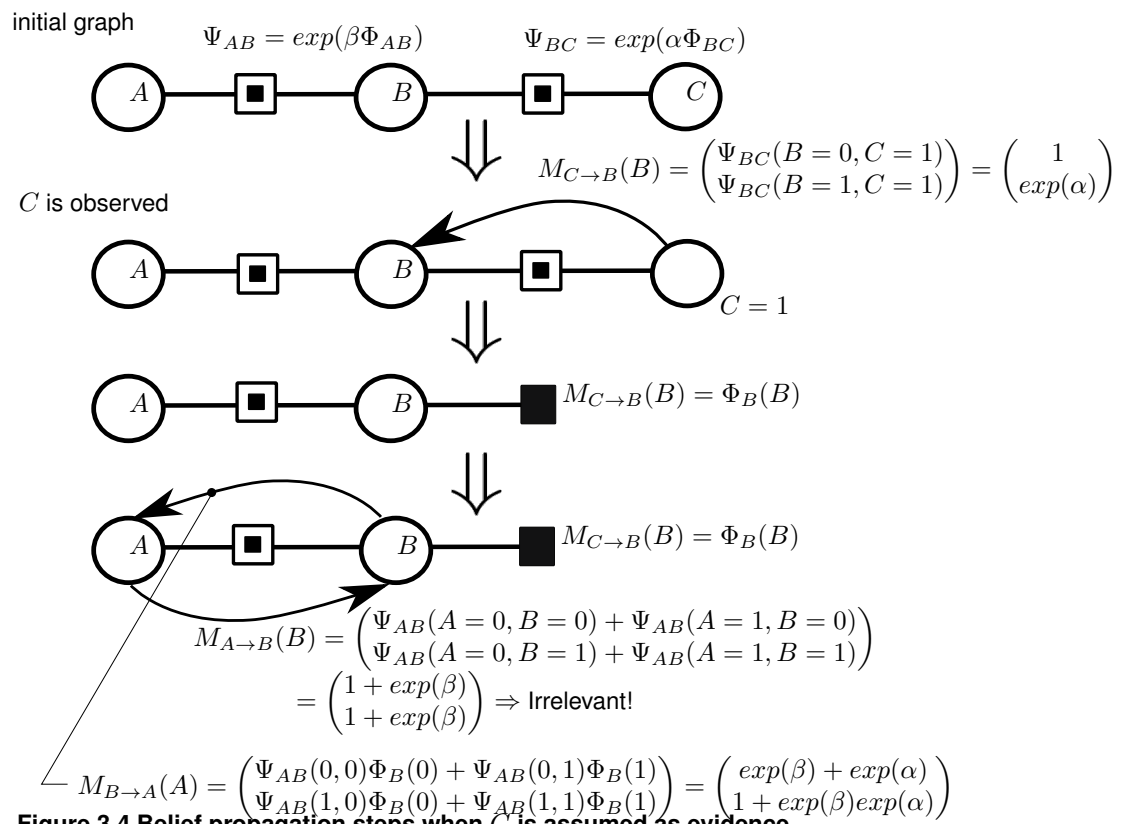


Figure 3.4 Belief propagation steps when C is assumed as evidence.

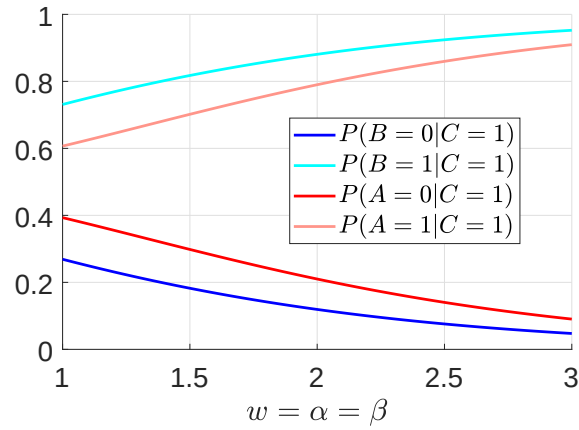


Figure 3.5 The marginals of variables B and A , when having a $C = 1$ as evidence of the graph reported in Figure 3.4.

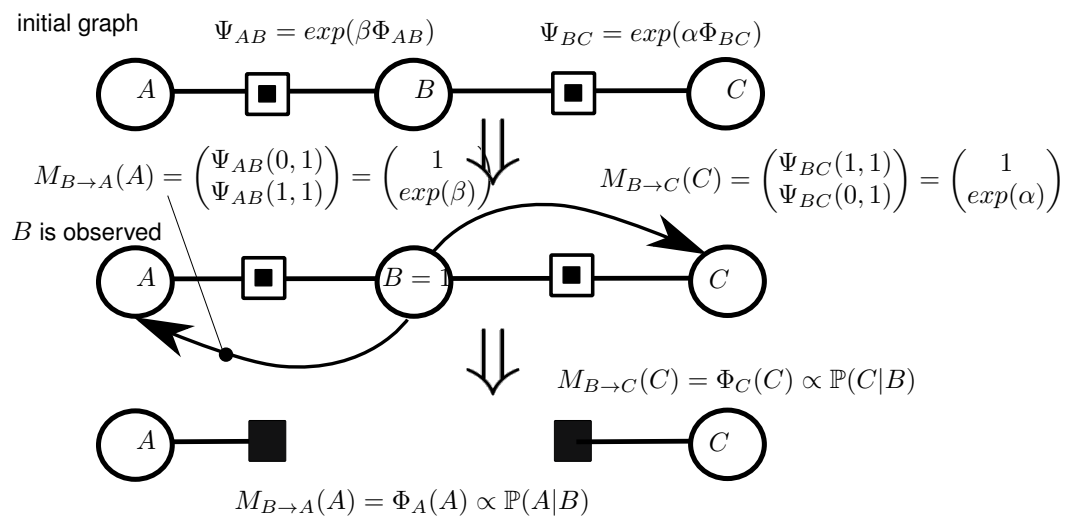
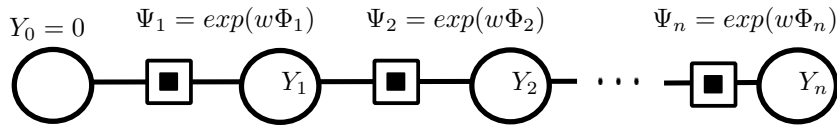


Figure 3.6 Belief propagation steps when B is assumed as evidence.



$\Psi_i(Y_{i-1}, Y_i)$	$Y_i = 0$	$Y_i = 1$	$\dots$	$Y_i = m$
$Y_{i-1} = 0$	$\exp(w)$	1	$\dots$	1
$Y_{i-1} = 1$	1	$\exp(w)$	$\dots$	1
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
$Y_{i-1} = m$	1	1	$\dots$	$\exp(w)$

Figure 3.7 On the top the chain considered in this example, while on the bottom the image of the generic factor $\Psi_i(Y_{i-1}, Y_i)$.

3.2.2 part 02

A slightly more complex graph, made of two exponential correlating factors Ψ_{BC} and Ψ_{AB} , is built in this sample. The considered graph is reported in Figure 3.3. The two involved factors have two different weights, α and β : the resulting image sets are reported in the right part of Figure 3.3.

In the first part, $C = 1$ is assumed as evidence and the marginal probabilities of A and B conditioned to C are computed. They are compared with the theoretical results, obtained by applying the message passing algorithm (Section 2.2.2), whose steps are here detailed ¹. The message passing steps are summarized in Figure 3.4. After having computed all the messages, it is clear that the marginal probabilities are equal to:

$$\mathbb{P}(A|C=1) = \frac{1}{Z} M_{B \rightarrow A}(A) = \frac{1}{Z} \left[\frac{\exp(\alpha) + \exp(\beta)}{1 + \exp(\alpha) \cdot \exp(\beta)} \right] \quad (3.15)$$

$$\mathbb{P}(B|C=1) = \frac{1}{Z} \Phi_B(B) \cdot M_{A \rightarrow B}(B) = \frac{1}{Z} \Phi_B(B) = \frac{1}{Z} \left[\frac{1}{\exp(\alpha)} \right] \quad (3.16)$$

Figure 3.5 shows the values assumed by the marginals when varying the coefficients α and β . As can be seen, the more A , B and C are correlated (i.e. the more α and β are big) the more $\mathbb{P}(B=1|C=1)$ and $\mathbb{P}(A=1|C=1)$ are big. Notice also that when assuming $\alpha = \beta$, $\mathbb{P}(B=1|C=1)$ is always bigger than $\mathbb{P}(A=1|C=1)$. This is intuitively explained by the fact that C is directly connected to B , while A is indirectly connected to C , through B .

In the second part, $B = 1$ is assumed as evidence and the marginal probabilities of A and C conditioned to B are computed. The theoretical results can be computed again considering the message passing, whose steps are reported in Figure 3.6. The marginal probabilities are in this case equal to:

$$\mathbb{P}(A|B=1) = \frac{1}{Z} \Phi_A(A) = \frac{1}{Z} \left[\frac{1}{\exp(\beta)} \right] \quad (3.17)$$

$$\mathbb{P}(C|B=1) = \frac{1}{Z} \Phi_C(C) = \frac{1}{Z} \left[\frac{1}{\exp(\alpha)} \right] \quad (3.18)$$

3.2.3 part 03

In this sample, a linear chain of variables $Y_{0,1,2,\dots,n}$ is considered. All variables in the chain have the same *Dom* size and all the factors $\Psi_{1,\dots,n}$, Figure 3.7, are simple exponential correlating factors. The image of the generic factor Ψ_i is reported in the right part of Figure 3.7.

The evidence $Y_0 = 0$ is assumed and the marginals of the last variable in the chain Y_n , i.e. the one furthest to Y_0 , are computed. Figure 3.8 reports the probability $\mathbb{P}(Y_n = 0|Y_0 = 0)$, when varying the chain size, as well the domain size of the variables. As can be seen, the more the chain is longer, the lower is the aforementioned probability, as Y_n is more and more indirectly correlated to Y_0 . Similar considerations hold for the domain size.

¹ The same steps are internally executed by Node_factory.

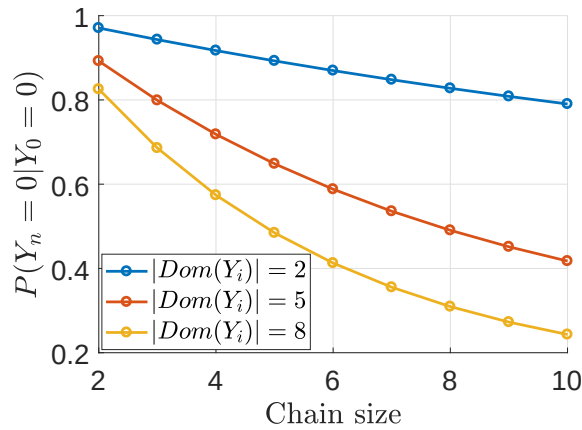


Figure 3.8 Marginal probability of Y_n when varying the chain size of the structure presented in Figure 3.7. w is assumed equal to 3.5.

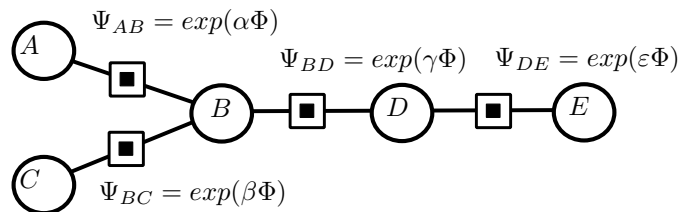


Figure 3.9 The factor graph considered by part 01.

3.3 Sample 03: Belief propagation, part B

The aim of this series of examples is to show how to perform probabilistic queries on articulated complex graphs.

3.3.1 part 01

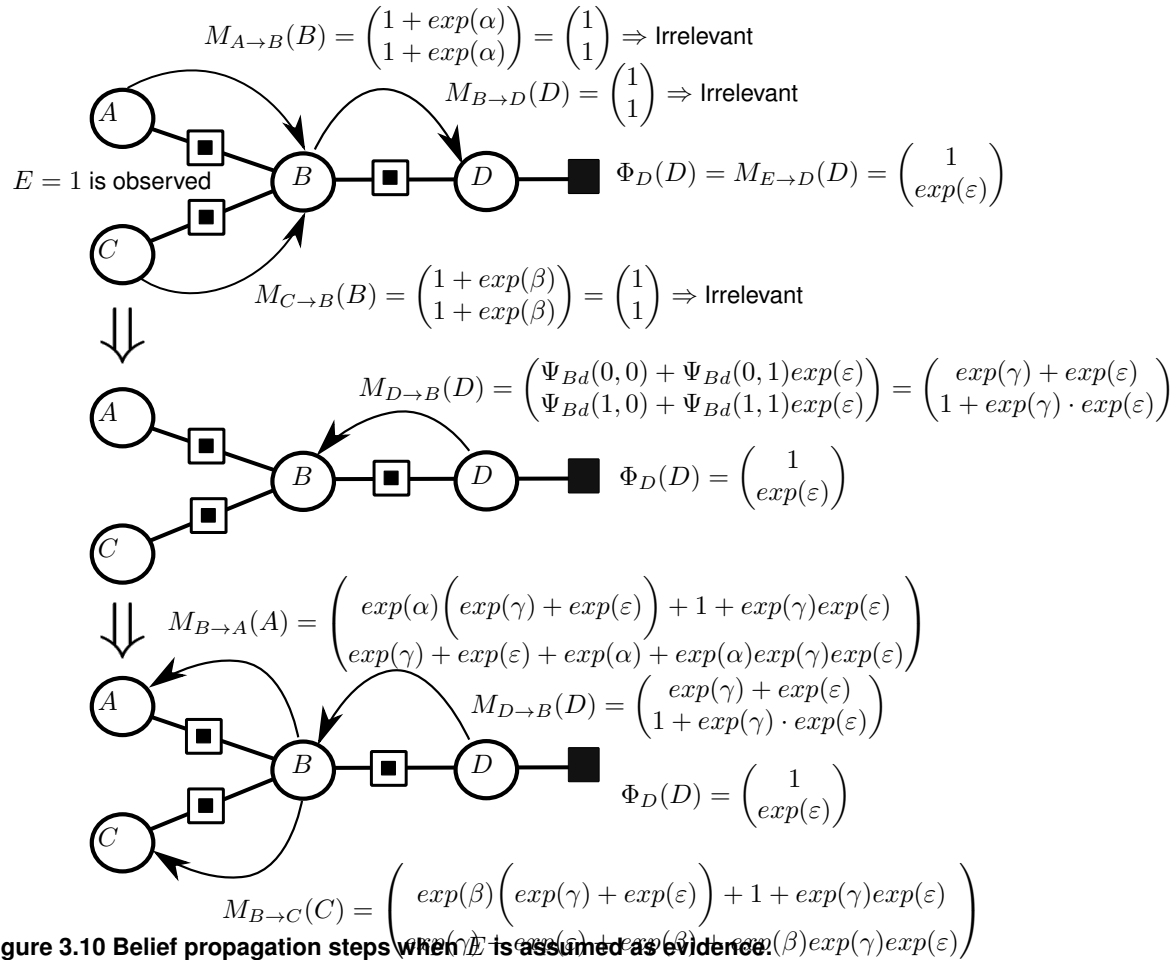
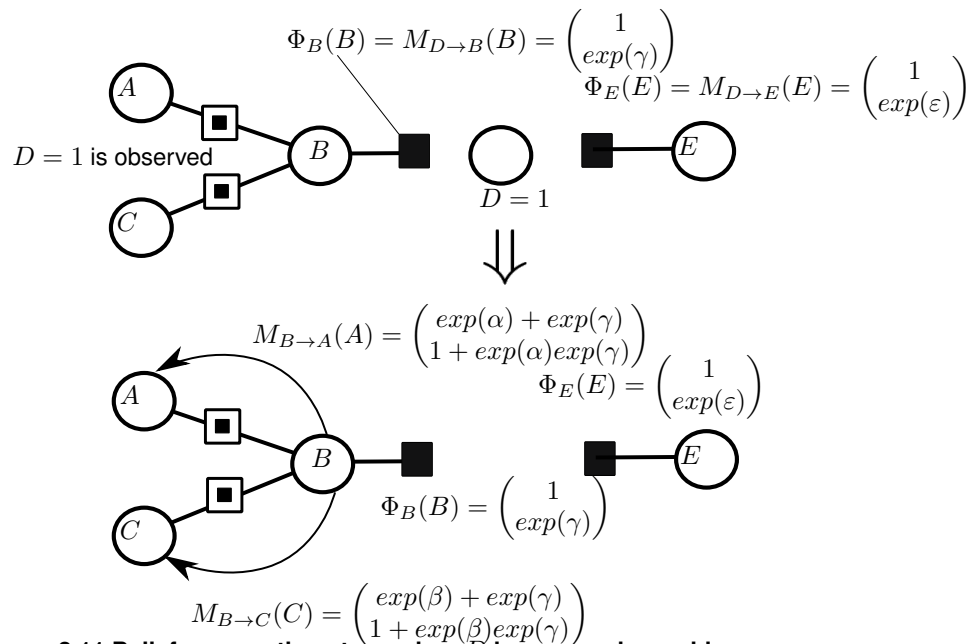
Part 01 considers a graph made of 5 variables with a Dom size equal to 2 and some simple exponential correlating factors, having different weights. The graph is reported in Figure 3.9, together with the weights of factor $\alpha, \beta, \gamma, \epsilon$. At first stage, the evidence $E = 1$ is assumed and the marginal probabilities of the other variables are computed with the message passing, whose steps are summarized in Figure 3.10. After the convergence of the message passing, the marginals of the variables are computed as similarly done for the previous examples. In a second phase, the evidence $E = 0$ is assumed. The computation of the marginals is omitted since it is specular to the previous case.

Finally, $D = 1$ is assumed and a new belief propagation is done, whose computations are reported in Figure 3.11.

3.3.2 part 02

Part 02 considers the graph reported in Figure 3.12. All the variables in Figure 3.12 have a domain size equal to 2, and all the factors are simply correlating exponential shape, having a unitary weight. Variables v_1, v_2 and v_3 are treated as evidences and the belief is propagated across the other ones, leading to the computation of the individual marginal probabilities. Since, the addressed structure is a politree (refer to Figure 2.6), the message passing algorithm converges within a finite number of steps.

In principle, the same approach followed in the previous examples can be followed to compute some theoretical results, with the aim of performing the comparisons. Anyway, for this kind of graph such an approach would be too complex. For this reason, results are compared with a Gibbs sampling approach: a series of samples $\mathcal{T} = \{T_1, \dots, T_N\}$ are taken from the joint conditioned distribution $\mathbb{P}(T = v_{4,5,6,7,8,9,10,11,12,13} | v_{1,2,3})$. Then, to

Figure 3.10 Belief propagation steps when $E = 1$ is assumed as evidence.Figure 3.11 Belief propagation steps when $D = 1$ is assumed as evidence.

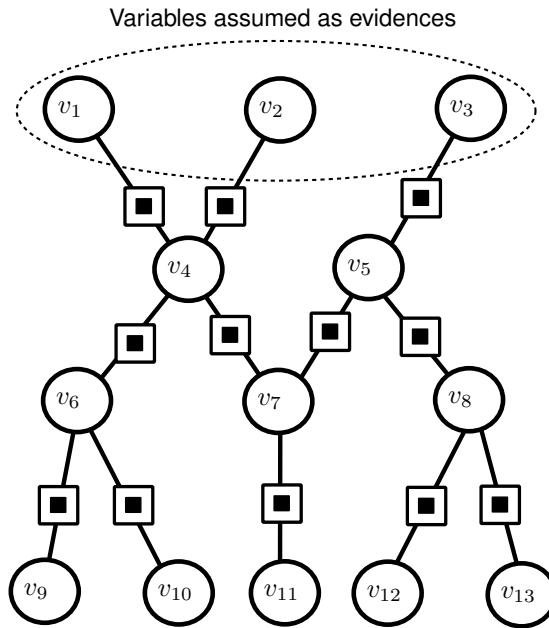


Figure 3.12 The factor graph considered by part 02.

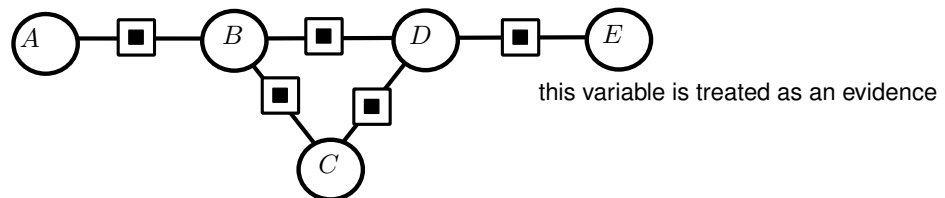


Figure 3.13 The factor graph considered by part 03.

evaluate the marginal probability $\mathbb{P}(v_i | v_{1,2,3})$ of a generic hidden variable v_i , the following empirical frequency is computed:

$$\mathbb{P}(v_i = v | v_{1,2,3}) = \frac{\sum_{T_j \in \mathcal{T}} L_{Ti}(T_j, v)}{N} \quad (3.19)$$

where $L_{Ti}(T_j, v)$ is an indicator function equal to 1 only for those samples for which v_i assumed a value equal to v .

3.3.3 part 03

Part 03 considers the graph reported in Figure 3.13. As for the example in the previous part, all variables are binary, and the potentials are simply exponential correlating with a unitary weight. However, this structure is loopy. E is treated as an evidence and the belief propagation is performed considering the loopy version of the message passing discussed in Section 2.2.2.

3.3.4 part 04

The last example in this series, considers a complex loopy graph, represented in Figure 3.14. As for other examples, all the variables are binary and the factors are exponential simply correlating with unitary weights. v_1 is assumed as evidence and the belief is propagated with the loopy version of message passing. Results are compared to the empirical frequencies obtained with a Gibbs sampler, as similarly done for the example of part 02.

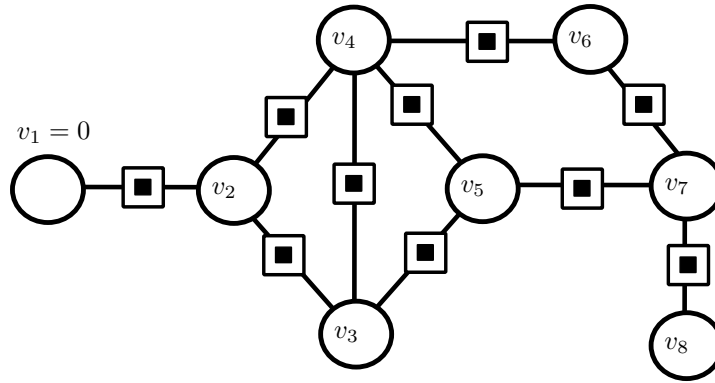


Figure 3.14 The factor graph considered by part 04.

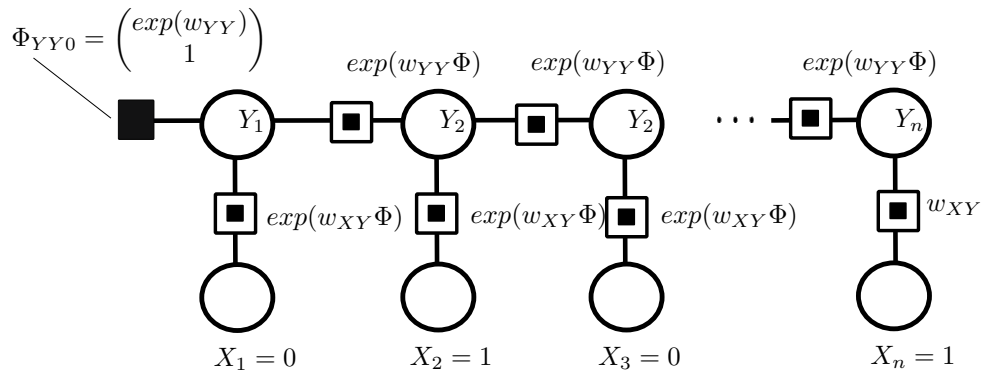


Figure 3.15 The chain structure considered by Sample 04.

3.4 Sample 04: Hidden Markov model like structure

The structure reported in Figure 3.15 is considered in this example. The reported chain is similar to those considered in Hidden Markov models, for which the chain of hidden variables $Y_{1,2,\dots}$ are connected to the chain of evidences $X_{1,2,\dots}$. The potential Φ_{YY0} , represents an a-priori knowledge about variable Y_1 . All the variables are binary and the potentials are simply correlating exponential potentials. In particular, the ones connecting the hidden variables have a weight equal to w_{YY} , while the ones connecting the evidences to the hidden set share a weight equal to w_{XY} . The evidences are set as indicated in Figure 3.15, i.e. 0 and 1 are alternated in the chain represented by $X_{1,2,\dots}$. The MAP estimation² of the hidden set (Section 2.3) is computed into two different situations:

- case a): $w_{XY} \gg w_{YY}$
- case b): $w_{XY} \ll w_{YY}$

Here the point is that when considering case a), the information about the evidences and the correlations between $Y_{1,2,\dots}$ and $X_{1,2,\dots}$ is predominant. On the opposite, when dealing with case b), the correlations among the hidden variables as well as the prior knowledge about Y_0 is predominant. For this reason, for case a) the MAP estimation of the hidden variables is equal to $h_{MAP}^a = \{0, 1, 0, 1, \dots\}$, while for case b) the MAP estimation is equal to $h_{MAP}^b = \{0, 0, 0, 0, \dots\}$.

3.5 Sample 05: Matricial structure

The matrix-like structure reported in Figure 3.16 is considered in this example. The variables in the matrix have all the same domain size and the are correlated by the potentials populating the matrix, which are all simple exponential

²The Node_factory class is in charge of invoking the proper version of the message passing algorithm that leads to the MAP estimation computation.

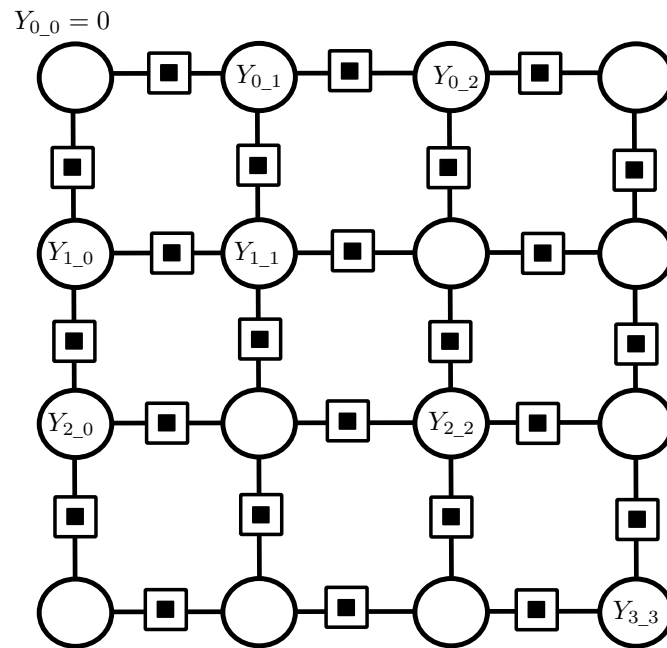


Figure 3.16 The matricial structure considered by Sample 05.

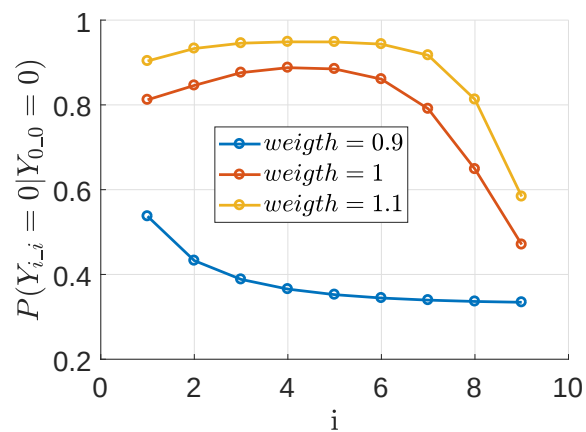


Figure 3.17 The marginals of variables $Y_{i,i}$, conditioned to $Y_{0,0} = 0$ as evidence of the graph reported in Figure 3.16, when varying the weight of the correlating exponential factors involved in the structure.

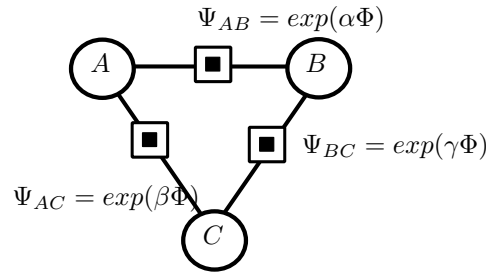


Figure 3.18 The graph considered by part 01.

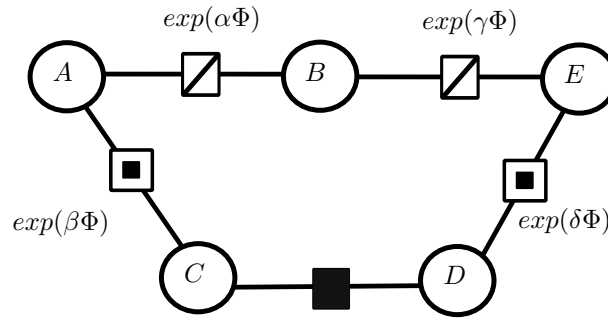


Figure 3.19 The graph considered by part 02.

correlating factors sharing the same weight. The example builds the matrix and then assumes $Y_{0_0} = 0$ as an evidence. Then, the marginals of the variables along the diagonal of the matrix, i.e. Y_{i_i} , are evaluated. As can be seen, the marginal probability $\mathbb{P}(Y_{i_i} = 0 | Y_{0_0})$ decreases descending the diagonal. Figure 3.17 reports the results obtained when varying the weight of the correlating factors, on a matrix made of 10×10 variables having a *Dom* size equal to 3.

3.6 Sample 06: Learning, part A

The aim of this series of examples is to show how to perform the learning of factor graphs. In all the examples contained in this Section, learning is done with the following methodology. A Gibbs sampler is used to take samples from the joint distribution correlating all the variables in a model A. Model A is actually the model to learn. The training set obtained from model A, is used to train a model B. Model B has the same variables and factors of model A, but with different values for the free parameters $w_{1,2,\dots}$ (Section 2.6). In this way, after having performed the learning, the value of the free parameters in model B will assume similar values to the ones in model A, showing the effectiveness of the functionalities contained in EFG. Clearly, this is not the approach followed in real applications, where the real coefficient of the model are unknown and only a training set of examples are available.

3.6.1 part 01

Part 01 considers the loopy graph reported in Figure 3.18. A , B and C are all binary variables, while Ψ_{AB} , Ψ_{AC} and Ψ_{BC} are simple correlating exponential distributions having as weights, respectively, α , β and γ . Some samples are generated from the model with a Gibbs sampling, in order to use it later as a training set. All the weights in the model are set to 1, and tuned using a quasi newton trainer that consider the previously generated training set. The aim is to show that after training the values of the weights are similar to the initial ones.

3.6.2 part 02

Part 02 considers a structure made of both tunable and non-tunable factors. The considered structure is reported in Figure 3.19. Weights β and δ must be tuned through learning, as similarly described in Section 3.6.1, while α and γ are constant and known (refer also to the formalism described in Figure 2.2).

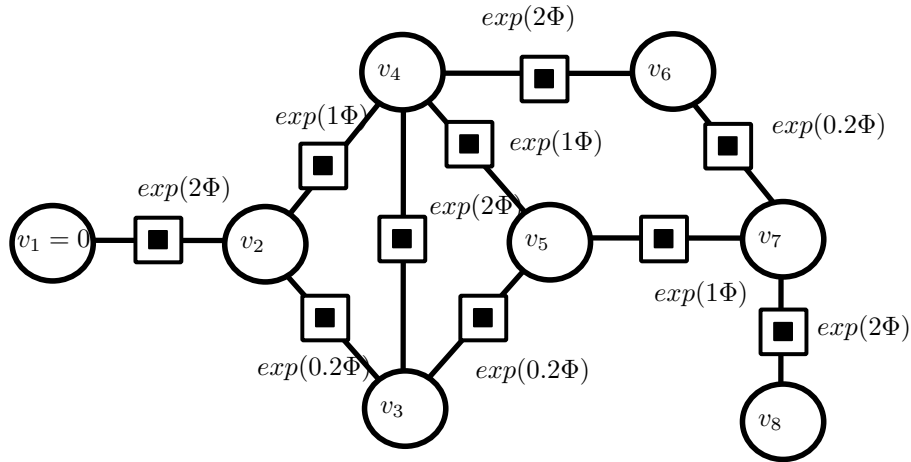


Figure 3.20 The graph considered by part 03.

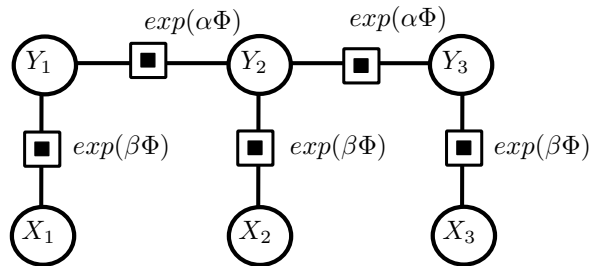


Figure 3.21 The graph considered by part 04.

3.6.3 part 03

Part 03 considers the loopy structure reported in Section 3.3.4. However, here instead of having constant exponential shapes, all the factors are made of tunable exponentials. The value assumed by the weight of model A (see the introduction of this Section) are showed in Figure 3.20. Weights are tuned through learning as similarly described in Section 3.6.1.

3.6.4 part 04

Part 04 considers the structure reported in Figure 3.21, for which all the potentials connecting pairs Y_{i-1}, Y_i share the same weight α , while the factors connecting pairs X_i, Y_i share the weight β . The approach described Weights are tuned through learning as similarly described in Section 3.6.1.

3.7 Sample 07: Learning, part B

The aim of this example is to show how the learning process can be done when dealing with Conditional random fields. In particular, the structure reported in Figure 3.22 is considered (values of the free parameters are not indicated, since the reader may refer to the sources provided).

The approach adopted is similar to the one followed in the previous series of example, considering a couple of model A and B (see the initial part of the previous Section). However, in this case we cannot simply draw samples from the joint distribution correlating the variables in the model, since such a distribution does not exists. Indeed, the conditional random field of Figure 3.22, models the conditional distribution of variables $Y_{1,2,\dots}$ w.r.t the evidences $X_{1,2,\dots}$. For this reason, all the possible combination of evidences are determined, considering all $x \in \{Dom(X_1) \cup Dom(X_2) \cup \dots\}$. For each x , samples from the conditioned distribution $\mathbb{P}(Y_{1,2,\dots}|x)$ are taken with a Gibbs sampler. The entire population of samples determined is actually the training set adopted fro training the conditional random field in Figure 3.22.

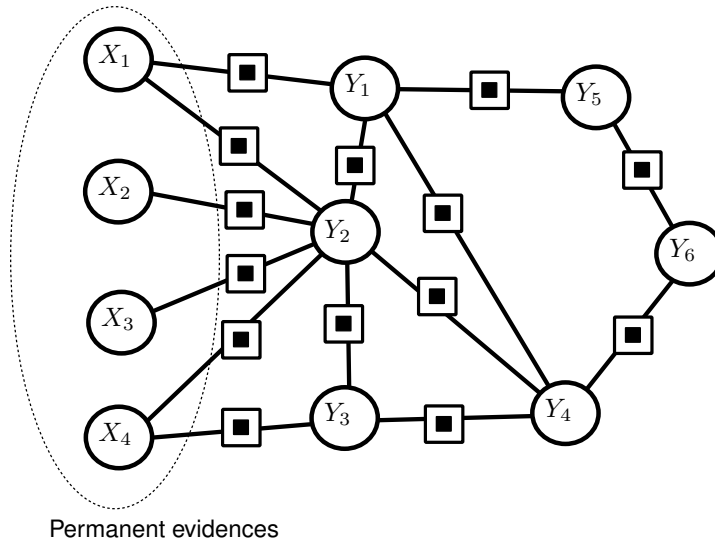


Figure 3.22 The conditional random field considered in Sample 07.

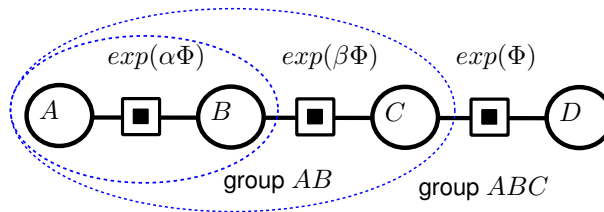


Figure 3.23 The chain considered in the example. All the underlying simple shapes are simple correlating.

3.8 Sample 08: Sub-graphing

3.8.1 part 01

The chain structure described in Figure 3.23 is addressed in this example. The sub-graphs containing variables A, B, C and A, B are built, in order to compute the joint marginal probability distributions of that two groups of variables.

The values are compared to the real ones, obtained by considering the joint distribution³ of all the variables in the chain, which can be obtained by computing the energy function E , equation (2.6) (computations are here omitted for brevity). Knowing the joint distribution of a group of variables, the marginal distribution of a sub-group can be obtained by marginalization, equation (2.4):

$$\begin{aligned}\mathbb{P}(A = a, B = b, C = c) &= \sum_{\tilde{d} \in \text{Dom}(D)} \mathbb{P}(A = a, B = b, C = c, D = \tilde{d}) \\ \mathbb{P}(A = a, B = b) &= \sum_{\tilde{c} \in \text{Dom}(C), \tilde{d} \in \text{Dom}(D)} \mathbb{P}(A = a, B = b, C = \tilde{c}, D = \tilde{d})\end{aligned}\quad (3.20)$$

Applying the above rules to the chain of Figure 3.23 leads to obtain the theoretical marginal distributions indicated in Figure 3.24.

3.8.2 part 02

The aim of this example is to show how sub-graphs (see Section 2.5) can be computed using SubGraph. The example starts building the structure described in Figure 3.25 (refer to the sources for the details regarding the

³Which is significantly time consuming to compute. For this reason, the SubGraph class is able to compute the marginals without explicitly compute the entire joint distribution of the variables in the model. Here we want just to compare the theoretical result with the one obtained by the SubGraph class.

A	B	$\mathbb{P}(A, B)$
0	0	$\frac{\exp(\alpha)}{1+\exp(\alpha)}$
0	1	$\frac{1}{1+\exp(\alpha)}$
1	0	$\frac{1}{1+\exp(\alpha)}$
1	1	$\frac{\exp(\alpha)}{1+\exp(\alpha)}$

A	B	C	$\mathbb{P}(A, B, C)$
0	0	0	$\frac{1}{Z_{ABC}} \cdot \exp(\alpha)\exp(\beta)$
0	1	0	$\frac{1}{Z_{ABC}}$
1	0	0	$\frac{1}{Z_{ABC}} \cdot \exp(\beta)$
1	1	0	$\frac{1}{Z_{ABC}} \cdot \exp(\alpha)$
0	0	1	$\frac{1}{Z_{ABC}} \cdot \exp(\alpha)$
0	1	1	$\frac{1}{Z_{ABC}} \cdot \exp(\beta)$
1	0	1	$\frac{1}{Z_{ABC}}$
1	1	1	$\frac{1}{Z_{ABC}} \cdot \exp(\alpha)\exp(\beta)$

Figure 3.24 Marginal probabilities of the sub-groups $\{A, B, C\}$ and $\{A, B\}$. The normalization coefficient

$$Z_{ABC} \text{ is equal to } Z_{ABC} = 2 \left(1 + \exp(\alpha) + \exp(\beta) + \exp(\alpha)\exp(\beta) \right).$$

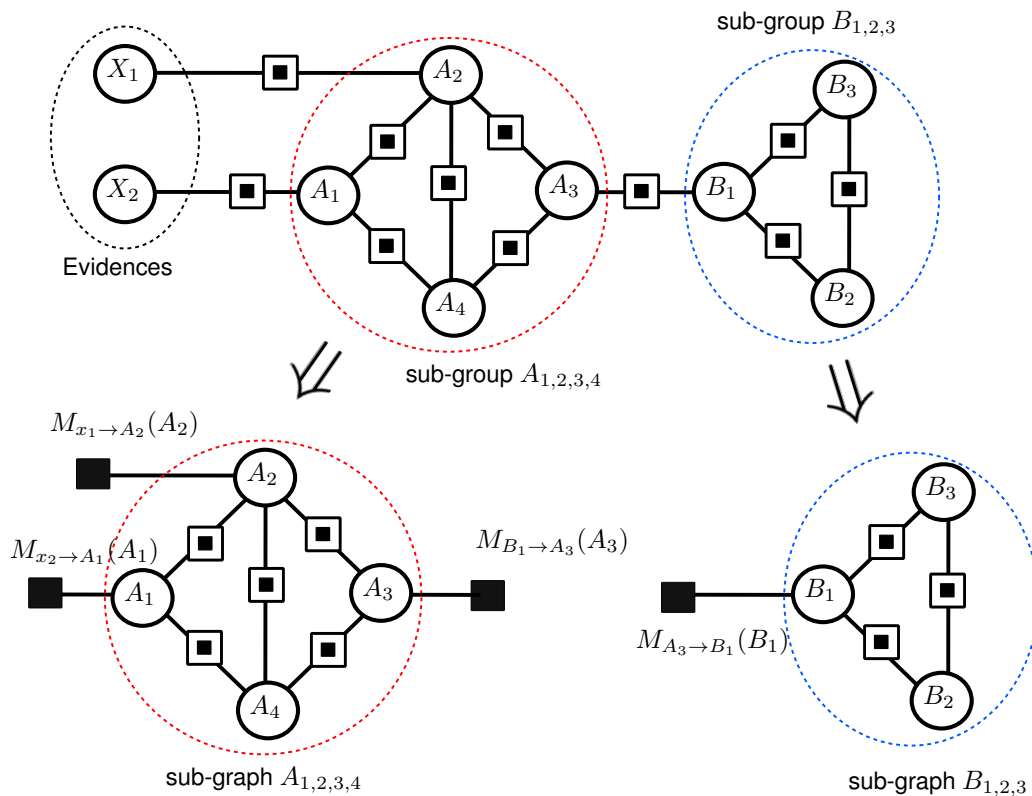


Figure 3.25 On the top, the graph considered by Sample 08, while on the bottom the sub-structures of the two groups $A_{1,2,3,4}$ and $B_{1,2,3}$.

variables and factors involved in the structure) and assumes the following evidences: $X_1 = 0$ and $X_2 = 0$. Then, the two sub-graphs considering the sub-group of variables $A_{1,2,3,4}$ and $B_{1,2,3}$ are computed, refer to Figure 3.25. The marginal probabilities of some combinations for $A_{1,2,3,4}$ conditioned to the evidences $X_{1,2}$ are computed and compared with the empirical frequencies computed considering a samples set produced by a Gibb sampler on the entire graph: samples for $t = A_{1,2,3,4}, B_{1,2,3}$ are drawn and the empirical frequencies of specific combinations of $A_{1,2,3,4}$ are computed as similarly done in 3.3.2. The same thing is done for the sub-graph $B_{1,2,3}$. At a second stage, the evidences $X_{1,2}$ are changed and the sub-graphs, as well as the marginal probabilities, are consequently recomputed.

Chapter 4

Namespace Index

4.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

EFG	47
EFG::categoric	47
EFG::factor	48
EFG::misc	50
EFG::model	51
EFG::structure	52

Chapter 5

Hierarchical Index

5.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

EFG::structure::BeliefManager	55
EFG::structure::Structure::BinaryFactorInStructure	55
EFG::misc::Chunk	56
EFG::misc::ConcurrentSafeLevels< T >	56
EFG::misc::ConcurrentSafeLevels< MessageMetaData::MessageIdx >	56
EFG::structure::Config	57
EFG::structure::ConfigManager	58
EFG::structure::ScopedEvidenceUpdate::Context	59
EFG::structure::Evidence	60
EFG::structure::EvidenceRemoveManager	60
EFG::model::ConditionalRandomField	56
EFG::structure::EvidenceSetManager	61
EFG::model::ConditionalRandomField	56
EFG::factor::ExponentialTrasform	62
EFG::structure::MessageMetaData::FactorInfo	63
EFG::factor::FactorT< N, Transform >	63
EFG::factor::FactorT< 1, Transform >	64
EFG::factor::FactorT< N, EFG::factor::ExponentialTrasform >	63
EFG::misc::GeneratorMap< U, T, GenT >	64
EFG::misc::GenFromBlock< T >	65
EFG::structure::GibbsSampler	65
EFG::categoric::GroupT< Support >	67
EFG::categoric::GroupT< N >	67
EFG::structure::HiddenCluster	67
EFG::misc::Intervals	67
EFG::misc::Intervals::IntervalsIter	68
EFG::misc::MemoryPool	68
EFG::structure::MessageMetaData	69
EFG::structure::ModelBuilder	70
EFG::structure::ModelExport	71
EFG::structure::ModelSeed	71
EFG::misc::ModeToStream< M >	71
EFG::misc::ModeToStream< FileMode::READ >	72
EFG::misc::ModeToStream< FileMode::WRITE >	72
EFG::structure::Trainer::NoCB	72

EFG::structure::Node	72
EFG::factor::NullTrasform	73
EFG::structure::PairHasher	73
EFG::structure::PropagationOrder	73
EFG::structure::PropagationResult	73
EFG::structure::QueryManager	74
EFG::misc::Range	75
EFG::categoric::RangeT< Support >	76
std::runtime_error	
EFG::Error	59
EFG::misc::Samples	77
EFG::structure::GibbsSampler::SamplesGenerationContext	77
EFG::misc::Samples::SamplesIter	78
EFG::structure::WorkerPoolManager::ScopedActivator	78
EFG::structure::ScopedEvidenceUpdate	79
EFG::factor::SimplyAntiCorrelatedDomainGen< N >	79
EFG::factor::SimplyCorrelatedDomainGen< N >	80
EFG::misc::Slot< T >	81
EFG::misc::Slot< float >	81
EFG::misc::Slot< IntervalPoint >	81
EFG::structure::Structure	81
EFG::structure::Trainer	82
EFG::structure::Tunability	83
EFG::structure::Tunability::TunableBinaryFactor	83
EFG::structure::Tunability::TunableUnaryFactor	83
EFG::structure::TunableWeightsManager::TunableWeightsGradient	84
EFG::structure::TunableWeightsManager	84
EFG::factor::UnaryFactorsMerger	84
EFG::structure::MessagePassingHandler	69
EFG::misc::UniformSampler	85
EFG::misc::ValuesPool	85
EFG::structure::ModelBuilder::VariableDescription	86
EFG::misc::VectorCache< Ts >	86
EFG::misc::VectorCache< EFG::structure::Evidence >	86
EFG::misc::VectorCache< float >	86
EFG::misc::VectorCache< float, float, float >	86
EFG::misc::VectorCache< std::size_t >	86
EFG::misc::VectorCache< std::size_t, float >	86
EFG::misc::VectorCache< T >	86
EFG::misc::VectorCache< Value >	86
EFG::misc::WorkerPool	86
EFG::structure::WorkerPoolActivityListener	87
EFG::structure::WorkerPoolManager	87
EFG::model::Components	
EFG::model::Model< Components >	70
EFG::model::ConditionalRandomField	56
EFG::model::Graph	66

Chapter 6

Class Index

6.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

EFG::structure::BeliefManager	55
EFG::structure::Structure::BinaryFactorInStructure	
Data	55
EFG::misc::Chunk	56
EFG::misc::ConcurrentSafeLevels< T >	56
EFG::model::ConditionalRandomField	56
EFG::structure::Config	57
EFG::structure::ConfigManager	58
EFG::structure::ScopedEvidenceUpdate::Context	59
EFG::Error	59
EFG::structure::Evidence	60
EFG::structure::EvidenceRemoveManager	60
EFG::structure::EvidenceSetManager	61
EFG::factor::ExponentialTrasform	62
EFG::structure::MessageMetaData::FactorInfo	63
EFG::factor::FactorT< N, Transform >	63
EFG::factor::FactorT< 1, Transform >	64
EFG::misc::GeneratorMap< U, T, GenT >	64
EFG::misc::GenFromBlock< T >	65
EFG::structure::GibbsSampler	
Refer also to https://en.wikipedia.org/wiki/Gibbs_sampling	65
EFG::model::Graph	
A simple graph object, that stores only const factors. Evidences may be changed over the time	66
EFG::categoric::GroupT< Support >	67
EFG::structure::HiddenCluster	67
EFG::misc::Intervals	67
EFG::misc::Intervals::IntervalsIter	68
EFG::misc::MemoryPool	68
EFG::structure::MessageMetaData	69
EFG::structure::MessagePassingHandler	69
EFG::model::Model< Components >	70
EFG::structure::ModelBuilder	70
EFG::structure::ModelExport	71
EFG::structure::ModelSeed	71
EFG::misc::ModeToStream< M >	71

EFG::misc::ModeToStream< FileMode::READ >	72
EFG::misc::ModeToStream< FileMode::WRITE >	72
EFG::structure::Trainer::NoCB	72
EFG::structure::Node	72
EFG::factor::NullTrasform	73
EFG::structure::PairHasher	73
EFG::structure::PropagationOrder	73
EFG::structure::PropagationResult	73
EFG::structure::QueryManager	74
EFG::misc::Range	75
EFG::categoric::RangeT< Support >	
This object allows to iterate all the elements in the joint domain of a group of variables, without precomputing all the elements in such domain. For example when having a domain made by variables = { A (size = 2), B (size = 3), C (size = 2) }, the elements in the joint domain that will be iterated are: <0,0,0> <0,0,1> <0,1,0> <0,1,1> <0,2,0> <0,2,1> <1,0,0> <1,0,1> <1,1,0> <1,1,1> <1,2,0> <1,2,1> After construction, the Range object starts to point to the first element in the joint domain <0,0,...>. Then, all other values can be iterated as with a normal iterator	76
EFG::misc::Samples	77
EFG::structure::GibbsSampler::SamplesGenerationContext	77
EFG::misc::Samples::SamplesIter	78
EFG::structure::WorkerPoolManager::ScopedActivator	78
EFG::structure::ScopedEvidenceUpdate	79
EFG::factor::SimplyAntiCorrelatedDomainGen< N >	
Similar to Factor(const categoric::Group &, const UseSimpleCorrelation &), but considering a simple anti-correlation. Therefore, to all combinations having all equal values, an image equal to 0 is assigned. All the other ones, are assigned a value equal to 1. For instance assume to pass a variable set equal to: {<A: size 2>, <B: size 2>}. Then, the following combinations map is built: <0,0,0> -> 0 <0,0,1> -> 1 <0,0,2> -> 1	79
EFG::factor::SimplyCorrelatedDomainGen< N >	
Describe a simple correlating domain. All the variables in the passed group should have the same size. The images pertaining to the combinations having all values equal, are assumed equal to 1, all the others to 0. For instance assume to pass a variable set equal to: {<A: size 3>, <B: size 3>, <C: size 3>}. Then, the following combinations map is built: <0,0,0> -> 1 <0,0,1> -> 0 <0,0,2> -> 0	80
EFG::misc::Slot< T >	81
EFG::structure::Structure	81
EFG::structure::Trainer	82
EFG::structure::Tunability	83
EFG::structure::Tunability::TunableBinaryFactor	83
EFG::structure::Tunability::TunableUnaryFactor	83
EFG::structure::TunableWeightsManager::TunableWeightsGradient	84
EFG::structure::TunableWeightsManager	84
EFG::factor::UnaryFactorsMerger	84
EFG::misc::UniformSampler	85
EFG::misc::ValuesPool	85
EFG::structure::ModelBuilder::VariableDescription	86
EFG::misc::VectorCache< Ts >	86
EFG::misc::VectorCache< T >	86
EFG::misc::WorkerPool	86
EFG::structure::WorkerPoolActivityListener	87
EFG::structure::WorkerPoolManager	87

Chapter 7

Namespace Documentation

7.1 EFG Namespace Reference

Namespaces

- [categoric](#)
- [factor](#)
- [misc](#)
- [model](#)
- [structure](#)

Classes

- class [Error](#)

7.1.1 Detailed Description

Author: Andrea Casalino Created: 01.01.2021

report any bug to andreca91@gmail.com.

7.2 EFG::categoric Namespace Reference

Classes

- class [GroupT](#)
- class [RangeT](#)

This object allows to iterate all the elements in the joint domain of a group of variables, without precomputing all the elements in such domain. For example when having a domain made by variables = { A (size = 2), B (size = 3), C (size = 2) }, the elements in the joint domain that will be iterated are: <0,0,0> <0,0,1> <0,1,0> <0,1,1> <0,2,0> <0,2,1> <1,0,0> <1,0,1> <1,1,0> <1,1,1> <1,2,0> <1,2,1> After construction, the Range object starts to point to the first element in the joint domain <0,0,...>. Then, all other values can be iterated as with a normal iterator.

Typedefs

- `template<std::size_t N>`
using **Group** = [GroupT](#)< Combination< N > >
- using **GroupVec** = [GroupT](#)< std::vector< VarStateSize > >
- `template<std::size_t N>`
using **Range** = [RangeT](#)< Combination< N > >
- using **RangeVec** = [RangeT](#)< std::vector< VarStateSize > >
- using **VarStateSize** = std::uint16_t
- `template<std::size_t N>`
using **Combination** = std::array< std::uint16_t, N >

Functions

- `template<std::size_t N, typename T >`
std::array< T, N > **make_same** (T val)

7.2.1 Detailed Description

Author: Andrea Casalino Created: 01.01.2021

report any bug to andreccasa91@gmail.com.

7.3 EFG::factor Namespace Reference

Classes

- struct [UnaryFactorsMerger](#)
- class [FactorT](#)
- class [FactorT](#)< 1, Transform >
- class [SimplyCorrelatedDomainGen](#)
Describe a simple correlating domain. All the variables in the passed group should have the same size. The images pertaining to the combinations having all values equal, are assumed equal to 1, all the others to 0. For instance assume to pass a variable set equal to: {<A: size 3>, <B: size 3>, <C: size 3>}. Then, the following combinations map is built: <0,0,0> -> 1 <0,0,1> -> 0 <0,0,2> -> 0.
- class [SimplyAntiCorrelatedDomainGen](#)
Similar to Factor(const categoric::Group &, const UseSimpleCorrelation &), but considering a simple anti-correlation. Therefore, to all combinations having all equal values, an image equal to 0 is assigned. All the other ones, are assigned a value equal to 1. For instance assume to pass a variable set equal to: {<A: size 2>, <B: size 2>}. Then, the following combinations map is built: <0,0,0> -> 0 <0,0,1> -> 1 <0,0,2> -> 1.
- struct [NullTrasform](#)
- struct [ExponentialTrasform](#)

Typedefs

- `template<std::size_t N>`
using **Factor** = [FactorT](#)< N, [NullTrasform](#) >
- using **UnaryFactor** = [FactorT](#)< 1, [NullTrasform](#) >
- using **BinaryFactor** = [FactorT](#)< 2, [NullTrasform](#) >
- `template<std::size_t N>`
using **FactorExponential** = [FactorT](#)< N, [ExponentialTrasform](#) >
- using **UnaryFactorExponential** = [FactorT](#)< 1, [ExponentialTrasform](#) >
- using **BinaryFactorExponential** = [FactorT](#)< 2, [ExponentialTrasform](#) >
- `template<std::size_t N>`
using **SparseDomainPoint** = std::pair< categoric::Combination< N >, float >

Functions

- `template<std::size_t N, typename Transform = NullTransform>`
`FactorT< N, Transform > make_simply_correlated (categoric::VarStateSize var_size)`
- `template<std::size_t N>`
`FactorExponential< N > make_exp_simply_correlated (categoric::VarStateSize var_size, float weight)`
- `template<std::size_t N, typename Transform = NullTransform>`
`FactorT< N, Transform > make_simply_anti_correlated (categoric::VarStateSize var_size)`
- `template<std::size_t N>`
`FactorExponential< N > make_exp_simply_anti_correlated (categoric::VarStateSize var_size, float weight)`
- `void normalize_probabilities (std::vector< float > &values)`
- `std::vector< float > make_probabilities (std::vector< float > values)`
- `template<std::size_t N, typename Transform >`
`void get_probabilities (const FactorT< N, Transform > &fctr, std::vector< float > &recipient)`
- `template<typename Transform >`
`factor::UnaryFactor make_evidence_message (std::span< float > message_support, const factor::FactorT< 2, Transform > &binary_factor, bool evidence_var_is_first, categoric::VarStateSize evidence_value)`
- `factor::UnaryFactor make_all_same (std::span< float > support, float value)`
- `factor::UnaryFactor make_indicator (std::span< float > support, categoric::VarStateSize value)`

7.3.1 Detailed Description

Author: Andrea Casalino Created: 31.03.2022

report any bug to andreca91@gmail.com.

Author: Andrea Casalino Created: 01.01.2021

report any bug to andreca91@gmail.com.

7.3.2 Function Documentation

7.3.2.1 `get_probabilities()`

```
template<std::size_t N, typename Transform >
void EFG::factor::get_probabilities (
    const FactorT< N, Transform > & fctr,
    std::vector< float > & recipient )
```

Returns

the probabilities associated to each combination in the domain, when assuming only the existence of this distribution. Such probabilities are actually the normalized images. The order of returned values, refer to the combinations that can be iterated by `categoric::GroupRange` on the variables representing this distribution.

7.4 EFG::misc Namespace Reference

Classes

- struct [ConcurrentSafeLevels](#)
- struct [ModeToStream](#)
- struct [ModeToStream](#)< [FileMode::READ](#) >
- struct [ModeToStream](#)< [FileMode::WRITE](#) >
- struct [GeneratorMap](#)
- struct [Range](#)
- class [GenFromBlock](#)
- struct [Chunk](#)
- class [Intervals](#)
- class [MemoryPool](#)
- struct [Slot](#)
- class [ValuesPool](#)
- class [Samples](#)
- class [UniformSampler](#)
- class [VectorCache](#)
- class [VectorCache](#)< T >
- class [WorkerPool](#)

Enumerations

- enum class [FileMode](#) { [READ](#) , [WRITE](#) }

Functions

- template<FileMode M>
std::unique_ptr< typename [ModeToStream](#)< M >::Stream > **open** (const std::filesystem::path &location)
- template<typename ArgFirst, typename... Args>
std::string **format** (std::string_view frmt, ArgFirst &&first, Args &&...args)
- template<typename T, typename GenT, typename Pred >
requires IsGenerator< GenT, T > void **for_each_generated** (GenT gen, Pred pred)
- template<typename T, typename GenT >
requires IsGenerator< GenT, T > std::vector< T > **materialize** (GenT gen)

Variables

- template<typename GenT, typename T >
concept **IsGenerator**

7.4.1 Detailed Description

Author: Andrea Casalino Created: 01.01.2021

report any bug to andreca91@gmail.com.

Author: Andrea Casalino Created: 31.03.2022

report any bug to andreca91@gmail.com.

7.4.2 Variable Documentation

7.4.2.1 IsGenerator

```
template<typename GenT , typename T >
concept EFG::misc::IsGenerator
```

Initial value:

```
= requires(GenT g) {
    { g.next() } -> std::convertible_to<std::optional<T>>;
}
```

7.5 EFG::model Namespace Reference

Classes

- class [ConditionalRandomField](#)
- class [Graph](#)
A simple graph object, that stores only const factors. Evidences may be changed over the time.
- class [Model](#)

Typedefs

- using [RandomField](#) = [Model](#)< [structure::BeliefManager](#), [structure::ConfigManager](#), [structure::EvidenceSetManager](#), [structure::EvidenceRemoveManager](#), [structure::GibbsSampler](#), [structure::QueryManager](#), [structure::WorkerPoolManager](#), [structure::TunableWeightsManager](#) `#ifdef EFG_JSON_IO, structure::ModelExport #endif` >
A simple graph object, that stores only const factors. Evidences may be changed over the time.

7.5.1 Detailed Description

Author: Andrea Casalino Created: 01.01.2021

report any bug to andreca91@gmail.com.

7.6 EFG::structure Namespace Reference

Classes

- class [MessagePassingHandler](#)
- class [BeliefManager](#)
- class [ConfigManager](#)
- struct [MessageMetaData](#)
- struct [PropagationOrder](#)
- struct [PropagationResult](#)
- struct [HiddenCluster](#)
- struct [Node](#)
- struct [Config](#)
- struct [Evidence](#)
- struct [ScopedEvidenceUpdate](#)
- class [EvidenceSetManager](#)
- class [EvidenceRemoveManager](#)
- class [GibbsSampler](#)

Refer also to https://en.wikipedia.org/wiki/Gibbs_sampling.

- class [ModelExport](#)
- struct [PairHasher](#)
- struct [Tunability](#)
- struct [ModelSeed](#)
- class [ModelBuilder](#)
- class [QueryManager](#)
- struct [WorkerPoolActivityListener](#)
- struct [Structure](#)
- class [TunableWeightsManager](#)
- struct [Trainer](#)
- class [WorkerPoolManager](#)

Typedefs

- `template<std::size_t N>`
using **FactorOrFactorExponential** = std::variant< [factor::FactorT](#)< N, [factor::NullTrasform](#) >, [factor::FactorT](#)< N, [factor::ExponentialTrasform](#) > >
- `template<std::size_t N>`
using **EmptyOrFactorOrFactorExponential** = std::variant< std::monostate, [factor::FactorT](#)< N, [factor::NullTrasform](#) >, [factor::FactorT](#)< N, [factor::ExponentialTrasform](#) > >
- using **PropagationLevels** = [misc::ConcurrentSafeLevels](#)< [MessageMetaData::MsgIdx](#) >
- using **HiddenClusters** = std::list< [HiddenCluster](#) >
- using **NamedVariables** = std::unordered_map< std::string, std::size_t >
- using **StructurePtr** = std::shared_ptr< [Structure](#) >

Enumerations

- enum class **PropagationKind** { **SUM** , **MAP** }

Functions

- `template<PropagationKind Kind, typename TransformT >`
`factor::UnaryFactor make_passed_message` (`std::span< float > message_support`, `std::span< const float > sender_merged_unaries`, `bool receiver_is_first`, `const factor::FactorT< 2, TransformT > &binary_factor`)
- `template<std::size_t N, typename Pred >`
`void use_factor_if` (`const EmptyOrFactorOrFactorExponential< N > &factor`, `Pred pred`)
- `structure::ModelSeed from_json` (`const nlohmann::json &json`)
- `structure::ModelSeed from_file` (`const std::filesystem::path &file_name`)
- `std::unique_ptr< misc::Samples > load_train_set` (`const std::filesystem::path &file_name`, `std::size_t samples_per_buffer`)
Imports the training set from a binary file. Combinations values are encoded as bytes train. Each line is interpreted as a sample.
- `void dump_train_set` (`const misc::Samples &samples`, `const std::filesystem::path &file_name`)

7.6.1 Detailed Description

Author: Andrea Casalino Created: 01.01.2021

report any bug to andreca91@gmail.com.

7.6.2 Function Documentation

7.6.2.1 load_train_set()

```
std::unique_ptr<misc::Samples> EFG::structure::load_train_set (
    const std::filesystem::path & file_name,
    std::size_t samples_per_buffer )
```

Imports the training set from a binary file. Combinations values are encoded as bytes train. Each line is interpreted as a sample.

Exceptions

<i>in</i>	case the passed file is inexistent
<i>in</i>	case not all the combinations in file have the same size.

Chapter 8

Class Documentation

8.1 EFG::structure::BeliefManager Class Reference

Public Member Functions

- `template<PropagationKind Kind>`
`void propagateBelief ()`

Protected Member Functions

- `void init (StructurePtr context)`

The documentation for this class was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/BeliefManager.h`

8.2 EFG::structure::Structure::BinaryFactorInStructure Struct Reference

data

```
#include <Structure.h>
```

Public Attributes

- `FactorOrFactorExponential< 2 > factor`
- `std::pair< std::size_t, std::size_t > vars_indices`

8.2.1 Detailed Description

data

The documentation for this struct was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/Structure.h`

8.3 EFG::misc::Chunk Struct Reference

Public Member Functions

- **Chunk** (std::size_t _begin, std::size_t _end, float _val)

Public Attributes

- std::size_t **begin**
- std::size_t **end**
- float **val**

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/Intervals.h

8.4 EFG::misc::ConcurrentSafeLevels< T > Struct Template Reference

Public Member Functions

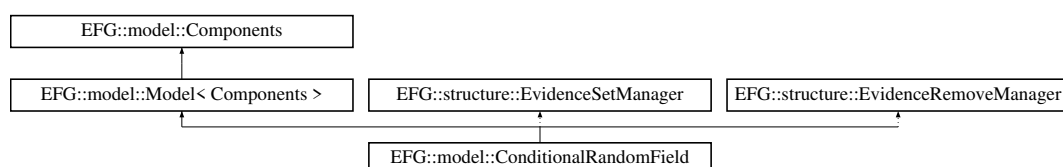
- template<typename Pred >
void **forEach** (Pred &&pred) const
- template<typename IterT >
void **add** (IterT begin, IterT end)
- template<typename Pred >
void **forEachLevel** (Pred pred) const

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/ConcurrentSafeLevels.h

8.5 EFG::model::ConditionalRandomField Class Reference

Inheritance diagram for EFG::model::ConditionalRandomField:



Public Member Functions

- **ConditionalRandomField** ([structure::ModelSeed](#) &&seed)
- void **setEvidences** (const std::vector< categoric::VarStateSize > &evidence_values)
- void **setEvidences** (std::span< const categoric::VarStateSize > evidence_values)
- [misc::Samples](#) **makeSamplesSpanningEvidences** (const SamplesGenerationContext &context)
Temporarily clear all evidences and samples from the model like it is RandomField with no evidences in an attempt to span the different possible observations values.

Additional Inherited Members

8.5.1 Member Function Documentation

8.5.1.1 makeSamplesSpanningEvidences()

```
misc::Samples EFG::model::ConditionalRandomField::makeSamplesSpanningEvidences (
    const SamplesGenerationContext & context )
```

Temporarily clear all evidences and samples from the model like it is RandomField with no evidences in an attempt to span the different possible observations values.

If you instead call makeSamples(...) (inherited from GibbsSampler), you are getting samples compatible with the last evidences set.

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/model/ConditionalRandomField.h

8.6 EFG::structure::Config Struct Reference

Public Attributes

- std::size_t **workers_pool_size** {0}
- std::size_t **max_calibration_iter** = 1000
- float **calibration_tolerance** = 1e-5

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/Definitions.h

8.7 EFG::structure::ConfigManager Class Reference

Public Member Functions

- void [setWorkersPoolSize](#) (std::size_t w)
- void [setMaxCalibrationIter](#) (std::size_t iters)
- void [setCalibrationTolerance](#) (float toll)
- const [Config](#) & [getconfig](#) () const

Protected Member Functions

- void [init](#) (StructurePtr context)

8.7.1 Member Function Documentation

8.7.1.1 [setCalibrationTolerance\(\)](#)

```
void EFG::structure::ConfigManager::setCalibrationTolerance (
    float toll ) [inline]
```

Parameters

<i>calibration</i>	is stopped whenever the maximum diff of message re-computation is below this threshold
--------------------	--

8.7.1.2 [setMaxCalibrationIter\(\)](#)

```
void EFG::structure::ConfigManager::setMaxCalibrationIter (
    std::size_t iters ) [inline]
```

Parameters

<i>The</i>	maximum number of iterations for doing model calibration
------------	--

8.7.1.3 [setWorkersPoolSize\(\)](#)

```
void EFG::structure::ConfigManager::setWorkersPoolSize (
    std::size_t w ) [inline]
```

Parameters

<i>The</i>	number of extra workers to spawn when training the model. When passing 0, the available number of cores on this machine is assumed.
------------	---

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/ConfigManager.h↔

8.8 EFG::structure::ScopedEvidenceUpdate::Context Struct Reference

Public Member Functions

- void **reset** ()

Public Attributes

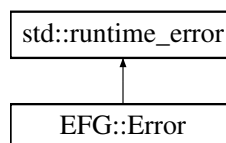
- std::vector< std::size_t > **vars_involved_by_update**
- std::unordered_map< [HiddenCluster](#) *, typename HiddenClusters::iterator > **clusters_involved_by_update**↔
- StructurePtr **structure**

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/EvidenceManager.h↔

8.9 EFG::Error Class Reference

Inheritance diagram for EFG::Error:



Public Member Functions

- **Error** (std::string what)
- template<typename ArgFirst, typename... Args>
Error (std::string_view frmt, ArgFirst &&first, Args &&...args)

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/Error.h

8.10 EFG::structure::Evidence Struct Reference

Public Attributes

- `std::size_t` **var_index**
- `categoric::VarStateSize` **value**

Static Public Attributes

- static const `categoric::VarStateSize` **NOT_AN_EVIDENCE**

8.10.1 Member Data Documentation

8.10.1.1 NOT_AN_EVIDENCE

```
const categoric::VarStateSize EFG::structure::Evidence::NOT_AN_EVIDENCE [inline], [static]
```

Initial value:

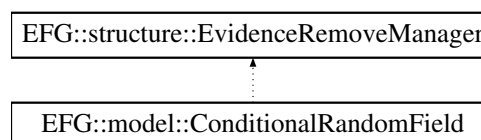
```
=
    std::numeric_limits<categoric::VarStateSize>::max()
```

The documentation for this struct was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/Definitions.h`

8.11 EFG::structure::EvidenceRemoveManager Class Reference

Inheritance diagram for EFG::structure::EvidenceRemoveManager:



Public Member Functions

- `template<typename... ARGS>`
void **removeEvidences** (`std::size_t` first, `ARGS` &&...others)
update the evidence set by removing the specified variable.
- void **removeEvidences** (`const std::vector< std::size_t >` &vars)
- void **removeAllEvidences** ()
removes all the evidences currently set for this model.

Protected Member Functions

- void **init** (StructurePtr context)

8.11.1 Member Function Documentation

8.11.1.1 removeEvidences()

```
template<typename...  ARGS>
void EFG::structure::EvidenceRemoveManager::removeEvidences (
    std::size_t first,
    ARGS &&...  others ) [inline]
```

update the evidence set by removing the specified variable.

Parameters

<i>the</i>	involved variable
------------	-------------------

Exceptions

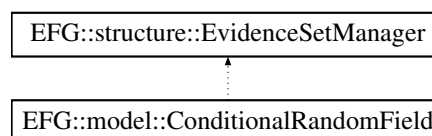
<i>in</i>	case the passed variable is not part of the model.
<i>in</i>	case the passed variable is not part of the current evidence set.

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/EvidenceRemoveManager.h

8.12 EFG::structure::EvidenceSetManager Class Reference

Inheritance diagram for EFG::structure::EvidenceSetManager:



Public Member Functions

- template<typename... ARGS>
void **setEvidences** ([Evidence](#) first, ARGS &&...others)
update the evidence set with the specified new evidence. In case the involved variable was already part of the evidence set, the evidence value is simply updated. On the contrary case, the involved variable is moved into the evidence set, with the specified value.
- void **setEvidences** (const std::vector< [Evidence](#) > &evidences)

Protected Member Functions

- void **init** (StructurePtr context)

8.12.1 Member Function Documentation

8.12.1.1 setEvidences()

```
template<typename...  ARGS>
void EFG::structure::EvidenceSetManager::setEvidences (
    Evidence first,
    ARGS &&...  others ) [inline]
```

update the evidence set with the specified new evidence. In case the involved variable was already part of the evidence set, the evidence value is simply updated. On the contrary case, the involved variable is moved into the evidence set, with the specified value.

Parameters

<i>the</i>	involved variable
<i>the</i>	evidence value

Exceptions

<i>in</i>	case the passed variable is not part of the model.
-----------	--

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/Evidence↵
Manager.h

8.13 EFG::factor::ExponentialTrasform Struct Reference

Public Member Functions

- float **operator()** (float val) const
- void **setWeight** (float w)
sets the weight used by the exponential evaluator.
- float **getWeight** () const

8.13.1 Member Function Documentation

8.13.1.1 getWeight()

```
float EFG::factor::ExponentialTrasform::getWeight ( ) const [inline]
```

Returns

the weight used by the exponential evaluator.

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/factor/Transform.h

8.14 EFG::structure::MessageMetaData::FactorInfo Struct Reference

Public Attributes

- bool **receiver_is_first_in_factor**
- std::size_t **receiver_index**
- std::size_t **sender_index**
- std::size_t **factor_index**

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/Definitions.h

8.15 EFG::factor::FactorT< N, Transform > Class Template Reference

Public Member Functions

- **FactorT** (categoric::Combination< N > sizes, [misc::Intervals](#) domain)
- template<bool UseTransformed>
float **get** (const categoric::Combination< N > &combination) const
- template<bool UseTransformed, typename Pred >
void **forEachCombination** (Pred pred) const
- template<bool UseTransformed>
void **getValues** (std::vector< float > &recipient) const
- const auto & **getVariables** () const
- void **transferIntoPool** ([misc::MemoryPool](#) &pool)
- std::size_t **getMemoryFootprint** () const
- template<bool Sort>
[FactorT](#)< N, Transform > **fromSparseDomain** (categoric::Combination< N > sizes, std::vector< Sparse↔
DomainPoint< N >> sparse_domain)
- template<typename G >
requires misc::IsGenerator< G, SparseDomainPoint< N > > [FactorT](#)< N, Transform > **fromSparse↔
DomainGen** (categoric::Combination< N > sizes, G gen)

Static Public Member Functions

- static [FactorT](#) **fromCompactDomain** (categoric::Combination< N > sizes, const std::vector< float > &compact_domain)
- template<bool Sort>
static [FactorT](#) **fromSparseDomain** (categoric::Combination< N > sizes, std::vector< SparseDomainPoint< N >> sparse_domain)
- template<typename G >
requires static misc::IsGenerator< G, SparseDomainPoint< N > > [FactorT](#) **fromSparseDomainGen** (categoric::Combination< N > sizes, G gen)

Public Attributes

- Transform **trsfm**

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/factor/FactorT.h

8.16 EFG::factor::FactorT< 1, Transform > Class Template Reference

Public Member Functions

- **FactorT** ([misc::Slot](#)< float > &&images)
- **FactorT** (const std::vector< float > &images)
- float **getTrsfm** (std::size_t index) const
- std::span< const float > **getAllValues** () const
- template<bool UseTransformed>
void **getValues** (std::vector< float > &recipient) const
- void **transferIntoPool** ([misc::MemoryPool](#) &pool)
- std::size_t **getMemoryFootprint** () const

Public Attributes

- Transform **trsfm**

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/factor/FactorT.h

8.17 EFG::misc::GeneratorMap< U, T, GenT > Struct Template Reference

Public Member Functions

- template<typename Pred >
GeneratorMap (GenT &&inner, Pred &&pred)
- std::optional< U > **next** ()

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/Generator.h

8.18 EFG::misc::GenFromBlock< T > Class Template Reference

Public Member Functions

- **GenFromBlock** (std::span< T > src)
- **GenFromBlock** (std::span< const T > src)
- **GenFromBlock** (const std::vector< T > &src)
- **GenFromBlock** (std::vector< T > &&src)
- std::optional< T > **next** ()

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/Generator.h

8.19 EFG::structure::GibbsSampler Class Reference

Refer also to https://en.wikipedia.org/wiki/Gibbs_sampling.

```
#include <GibbsSampler.h>
```

Classes

- struct [SamplesGenerationContext](#)

Public Member Functions

- [misc::Samples](#) **makeSamples** (const [SamplesGenerationContext](#) &context)
Use Gibbs sampling approach to draw empirical samples. Values inside the returned combination are ordered with the same order used for the variables returned by getAllVariables().

Protected Member Functions

- void **init** (StructurePtr context)

8.19.1 Detailed Description

Refer also to https://en.wikipedia.org/wiki/Gibbs_sampling.

8.19.2 Member Function Documentation

8.19.2.1 makeSamples()

```
misc::Samples EFG::structure::GibbsSampler::makeSamples (
    const SamplesGenerationContext & context )
```

Use Gibbs sampling approach to draw empirical samples. Values inside the returned combination are ordered with the same order used for the variables returned by getAllVariables().

In case some evidences are set, their values will appear as is in the sampled combinations.

Parameters

<i>number</i>	parameters for the samples generation
<i>number</i>	of threads to use for the samples generation

The documentation for this class was generated from the following file:

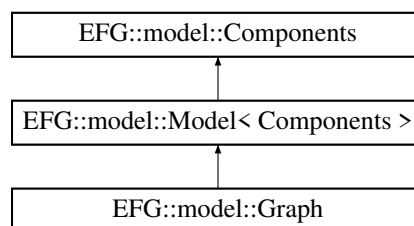
- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/GibbsSampler.h

8.20 EFG::model::Graph Class Reference

A simple graph object, that stores only const factors. Evidences may be changed over the time.

```
#include <Graph.h>
```

Inheritance diagram for EFG::model::Graph:



Public Member Functions

- **Graph** ([structure::ModelSeed](#) &&seed)

Additional Inherited Members

8.20.1 Detailed Description

A simple graph object, that stores only const factors. Evidences may be changed over the time.

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/model/Graph.h

8.21 EFG::categoric::GroupT< Support > Class Template Reference

Public Member Functions

- **GroupT** (Support sizes)
- const auto & **sizes** () const
- std::size_t **combinationsLen** () const
- std::size_t **combinationIndex** (const Support &combination) const
- Support **combinationFromIndex** (std::size_t idx) const
- bool **check** (const Support &combination) const

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/categoric/Group.h

8.22 EFG::structure::HiddenCluster Struct Reference

Public Attributes

- std::vector< std::size_t > **variables**
- std::optional< [PropagationOrder](#) > **order**
- std::optional< [PropagationResult](#) > **last_performed_propagation**

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/Definitions.h

8.23 EFG::misc::Intervals Class Reference

Classes

- struct [IntervalsIter](#)

Public Types

- using **IntervalPoint** = std::pair< std::size_t, float >

Public Member Functions

- float **get** (std::size_t index) const
- [IntervalsIter](#) **iter** () const
- const auto & **getDelimiters** () const
- void **transferIntoPool** ([misc::MemoryPool](#) &pool)
- template<typename G >
requires IsGenerator< G, Intervals::IntervalPoint > [Intervals](#) **fromGen** (G gen)
- template<typename G >
requires IsGenerator< G, [Chunk](#) > [Intervals](#) **fromGen** (G gen)

Static Public Member Functions

- `template<bool Sort>`
static [Intervals](#) **fromPoints** (std::vector< IntervalPoint > collection)
- `template<bool Sort>`
static [Intervals](#) **fromChunks** (std::vector< [Chunk](#) > collection)
- `template<typename G >`
requires static IsGenerator< G, IntervalPoint > [Intervals](#) **fromGen** (G gen)
- `template<typename G >`
requires static IsGenerator< G, [Chunk](#) > [Intervals](#) **fromGen** (G gen)

The documentation for this class was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/Intervals.h`

8.24 EFG::misc::Intervals::IntervalIter Struct Reference

Public Member Functions

- `float next ()`

Friends

- class [Intervals](#)

The documentation for this struct was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/Intervals.h`

8.25 EFG::misc::MemoryPool Class Reference

Public Member Functions

- **MemoryPool** (std::size_t capacity_limit)
- **MemoryPool** (const [MemoryPool](#) &)=delete
- [MemoryPool](#) & **operator=** (const [MemoryPool](#) &)=delete
- **MemoryPool** ([MemoryPool](#) &&) noexcept=delete
- [MemoryPool](#) & **operator=** ([MemoryPool](#) &&) noexcept=delete
- `template<typename T >`
std::span< T > **cloneIntoPool** (std::span< T > slot)
- `template<typename T >`
std::span< T > **claimFromPool** (std::size_t size)

The documentation for this class was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/MemoryPool.h`

8.26 EFG::structure::MessageMetaData Struct Reference

Classes

- struct [FactorInfo](#)

Public Types

- using **MsgIdx** = std::size_t

Public Attributes

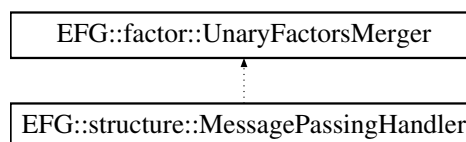
- bool **is_available** {false}
- MsgIdx **idx**
- MsgIdx **twin_idx**
- [FactorInfo](#) **factor_info**

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/Definitions.h

8.27 EFG::structure::MessagePassingHandler Class Reference

Inheritance diagram for EFG::structure::MessagePassingHandler:



Public Member Functions

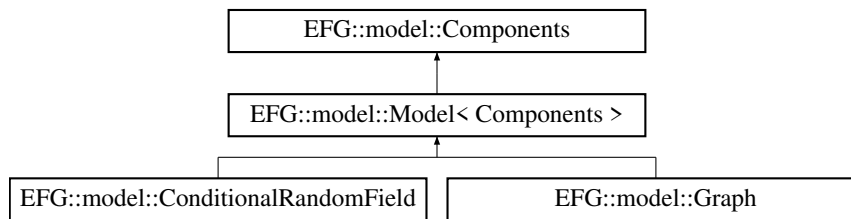
- template<PropagationKind Kind, bool EvalDiff>
float **compute_message** ([Structure](#) &context, [MessageMetaData](#) &msg)
- template<PropagationKind Kind, bool EvalDiff>
float **compute_message** ([Structure](#) &context, [MessageMetaData](#) &msg, std::span< float > recipient)

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/BeliefManager.h

8.28 EFG::model::Model< Components > Class Template Reference

Inheritance diagram for EFG::model::Model< Components >:



Public Member Functions

- **Model** ([structure::ModelSeed](#) &&seed)
- **Model** (const [Model](#) &)=delete
- [Model](#) & **operator=** (const [Model](#) &)=delete
- **Model** ([Model](#) &&) noexcept=delete
- [Model](#) & **operator=** ([Model](#) &&) noexcept=delete

Protected Member Functions

- auto **getContextPtr** ()

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/model/Model.h

8.29 EFG::structure::ModelBuilder Class Reference

Classes

- struct [VariableDescription](#)

Public Member Functions

- **ModelBuilder** (const [ModelBuilder](#) &)=delete
- [ModelBuilder](#) & **operator=** (const [ModelBuilder](#) &)=delete
- std::size_t **make_variable** (categoric::VarStateSize variable_size)
- std::size_t **make_variable** (categoric::VarStateSize variable_size, [VariableDescription](#) description)
- const auto & **get_seed** () const
- template<typename Trasform >
void **add_unary_factor** ([factor::FactorT](#)< 1, Trasform > &&to_add, std::size_t var_index)
- void **add_tunable_unary_factor** ([factor::UnaryFactorExponential](#) &&to_add, std::size_t var_index)
- template<typename Trasform >
void **add_binary_factor** ([factor::FactorT](#)< 2, Trasform > &&to_add, std::size_t var_a_index, std::size_t var_b_index)
- void **add_tunable_binary_factor** ([factor::BinaryFactorExponential](#) &&to_add, std::size_t var_a_index, std::size_t var_b_index)

Static Public Member Functions

- static [ModelSeed](#) **build** ([ModelBuilder](#) &&builder)

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/ModelSeed.h

8.30 EFG::structure::ModelExport Class Reference

Public Member Functions

- void **to_json** (nlohmann::json &json) const
- void **to_file** (const std::filesystem::path &file_name) const

Protected Member Functions

- void **init** (structure::StructurePtr context)

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/ModelExport.h

8.31 EFG::structure::ModelSeed Struct Reference

Public Attributes

- std::vector< categoric::VarStateSize > **variables_sizes**
- std::unordered_map< std::size_t, FactorOrFactorExponential< 1 > > **unary_factors**
- std::unordered_map< std::pair< std::size_t, std::size_t >, FactorOrFactorExponential< 2 >, [PairHasher](#) > **binary_factors**
- NamedVariables **named**
- std::vector< std::size_t > **permanent_evidences**
- [Tunability](#) **tunability**

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/ModelSeed.h

8.32 EFG::misc::ModeToStream< M > Struct Template Reference

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/File.h

8.33 EFG::misc::ModeToStream< FileMode::READ > Struct Reference

Public Types

- using **Stream** = std::ifstream

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/File.h

8.34 EFG::misc::ModeToStream< FileMode::WRITE > Struct Reference

Public Types

- using **Stream** = std::ofstream

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/File.h

8.35 EFG::structure::Trainer::NoCB Struct Reference

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/Tunable↔
WeightsManager.h

8.36 EFG::structure::Node Struct Reference

Public Attributes

- categoric::VarStateSize **var_size**
- categoric::VarStateSize **evidence**
- EmptyOrFactorOrFactorExponential< 1 > **unary_factor**
- std::span< [MessageMetaData](#) > **incoming_messages**
- HiddenClusters::iterator **hidden_cluster_it**

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/Definitions.h

8.37 EFG::factor::NullTransform Struct Reference

Public Member Functions

- float **operator()** (float val) const

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/factor/Transform.h

8.38 EFG::structure::PairHasher Struct Reference

Public Member Functions

- std::size_t **operator()** (const std::pair< std::size_t, std::size_t > &p) const

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/ModelSeed.h

8.39 EFG::structure::PropagationOrder Struct Reference

Public Attributes

- [PropagationLevels](#) **deterministic_part**
- std::vector< MessageMetaData::Messageldx > **to_calibrate_part**

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/Definitions.h

8.40 EFG::structure::PropagationResult Struct Reference

Public Attributes

- PropagationKind **propagation_kind_done**
- bool **was_completed**
- std::size_t **iterations**

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/Definitions.h

8.41 EFG::structure::QueryManager Class Reference

Public Member Functions

- void [getMarginalDistribution](#) (std::vector< float > &recipient, std::size_t variable_index)
- void [getJointMarginalDistributionProbabilities](#) (std::vector< float > &recipient, const std::vector< std::size_t > &vars)
- template<typename... ARGS>
void [getJointMarginalDistributionProbabilities](#) (std::vector< float > &recipient, std::size_t var_first, std::size_t var_second, ARGS &&...others)
- template<std::size_t N>
[factor::FactorT](#)< N, [factor::NullTransform](#) > [getJointMarginalDistribution](#) (const std::array< std::size_t, N > &vars)
- categoric::VarStateSize [getMAP](#) (std::size_t variable_index)
- void [getHiddenSetMAP](#) (std::vector< categoric::VarStateSize > &recipient)
- const auto & [getStructure](#) () const

Protected Member Functions

- void [init](#) (StructurePtr context)

8.41.1 Member Function Documentation

8.41.1.1 [getHiddenSetMAP\(\)](#)

```
void EFG::structure::QueryManager::getHiddenSetMAP (
    std::vector< categoric::VarStateSize > & recipient )
```

Returns

the Maximum a Posteriori estimation of the hidden variables, conditioned to the last set of evidences. Values are ordered with the same order used by the set of variables returned in [getHiddenVariables\(\)](#)

Parameters

<i>the</i>	number of threads to use for propagating the belief before returning the result.
------------	--

8.41.1.2 [getMAP\(\)](#)

```
categoric::VarStateSize EFG::structure::QueryManager::getMAP (
    std::size_t variable_index ) [inline]
```

Returns

the Maximum a Posteriori estimation of a specific variable in the model, conditioned to the last set of evidences.

Parameters

<i>the</i>	involved variable
<i>the</i>	number of threads to use for propagating the belief before returning the result.

Exceptions

<i>when</i>	the passed variable name is not found
-------------	---------------------------------------

8.41.1.3 getMarginalDistribution()

```
void EFG::structure::QueryManager::getMarginalDistribution (
    std::vector< float > & recipient,
    std::size_t variable_index ) [inline]
```

Parameters

<i>the</i>	marginal probability of the passed variable, i.e. $P(\text{var} \text{observations})$, conditioned to the last set of evidences.
<i>the</i>	involved variable
<i>the</i>	number of threads to use for propagating the belief before returning the result.

Exceptions

<i>when</i>	the passed variable name is not found
-------------	---------------------------------------

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/QueryManager.h↔

8.42 EFG::misc::Range Struct Reference

Public Member Functions

- **Range** (std::size_t len)
- std::optional< std::size_t > **next** ()

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/Generator.h

8.43 EFG::categoric::RangeT< Support > Class Template Reference

This object allows to iterate all the elements in the joint domain of a group of variables, without precomputing all the elements in such domain. For example when having a domain made by variables = { A (size = 2), B (size = 3), C (size = 2) }, the elements in the joint domain that will be iterated are: <0,0,0> <0,0,1> <0,1,0> <0,1,1> <0,2,0> <0,2,1> <1,0,0> <1,0,1> <1,1,0> <1,1,1> <1,2,0> <1,2,1> After construction, the Range object starts to point to the first element in the joint domain <0,0,...>. Then, all other values can be iterated as with a normal iterator.

```
#include <Range.h>
```

Public Member Functions

- [RangeT](#) (Support &&sizes)
- `std::optional< const Support * > next ()`

8.43.1 Detailed Description

```
template<typename Support>
class EFG::categoric::RangeT< Support >
```

This object allows to iterate all the elements in the joint domain of a group of variables, without precomputing all the elements in such domain. For example when having a domain made by variables = { A (size = 2), B (size = 3), C (size = 2) }, the elements in the joint domain that will be iterated are: <0,0,0> <0,0,1> <0,1,0> <0,1,1> <0,2,0> <0,2,1> <1,0,0> <1,0,1> <1,1,0> <1,1,1> <1,2,0> <1,2,1> After construction, the Range object starts to point to the first element in the joint domain <0,0,...>. Then, all other values can be iterated as with a normal iterator.

This object should be recognized by the compiler as an stl iterator.

8.43.2 Constructor & Destructor Documentation

8.43.2.1 RangeT()

```
template<typename Support >
EFG::categoric::RangeT< Support >::RangeT (
    Support && sizes )
```

Parameters

<i>sizes</i>	of the variables this group represents
--------------	--

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/categoric/Range.h

8.44 EFG::misc::Samples Class Reference

Classes

- struct [SamplesIter](#)

Public Member Functions

- **Samples** (std::size_t sample_len, std::size_t samples_per_buffer=1000)
- **Samples** (const [Samples](#) &)=delete
- **Samples** & **operator=** (const [Samples](#) &)=delete
- **Samples** ([Samples](#) &&o) noexcept
- **Samples** & **operator=** ([Samples](#) &&o) noexcept
- void **add** (std::span< const categoric::VarStateSize > vals)
- void **add** (const std::vector< categoric::VarStateSize > &vals)
- [SamplesIter](#) **makeliter** () const
- std::size_t **eachSampleSize** () const
- std::size_t **samplesCount** () const

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/Samples.h

8.45 EFG::structure::GibbsSampler::SamplesGenerationContext Struct Reference

Public Attributes

- std::size_t **samples_number**
- std::optional< std::size_t > [delta_iterations](#)
number of iterations used to evolve the model between the drawing of one sample and another
- std::optional< std::size_t > [seed](#)
sets the seed of the random engine. Passing a nullopt will make the sampler to generate a random seed by using the current time.
- std::optional< std::size_t > [transient](#)
number of samples to discard before actually starting the sampling procedure.
- bool **include_evidences_in_samples** {false}

8.45.1 Member Data Documentation

8.45.1.1 [delta_iterations](#)

```
std::optional<std::size_t> EFG::structure::GibbsSampler::SamplesGenerationContext::delta_↵
iterations
```

number of iterations used to evolve the model between the drawing of one sample and another

When nothing is specified, $0.1 * \text{samples_number}$ is assumed

8.45.1.2 transient

`std::optional<std::size_t> EFG::structure::GibbsSampler::SamplesGenerationContext::transient`

number of samples to discard before actually starting the sampling procedure.

When nothing is specified, 10 times `delta_iterations` is assumed.

The documentation for this struct was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/GibbsSampler.h`

8.46 EFG::misc::Samples::SamplesIter Struct Reference

Public Member Functions

- **SamplesIter** (const [Samples](#) &source)
- `std::optional< std::span< const categoric::VarStateSize > > next ()`

The documentation for this struct was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/Samples.h`

8.47 EFG::structure::WorkerPoolManager::ScopedActivator Struct Reference

Public Member Functions

- **ScopedActivator** (const [ScopedActivator](#) &)=delete
- [ScopedActivator](#) & **operator=** (const [ScopedActivator](#) &)=delete
- **ScopedActivator** ([ScopedActivator](#) &&) noexcept=delete
- [ScopedActivator](#) & **operator=** ([ScopedActivator](#) &&) noexcept=delete
- [misc::WorkerPool](#) & **getWorkers** ()

Friends

- class **WorkerPoolManager**

The documentation for this struct was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/WorkerPool↵
Manager.h`

8.48 EFG::structure::ScopedEvidenceUpdate Struct Reference

Classes

- struct [Context](#)

Public Member Functions

- **ScopedEvidenceUpdate** ([Context](#) &ctxt)
- template<bool ValidateValue>
void **validateEvidence** (std::size_t variable_index, categoric::VarStateSize value)
- void **addCluster** (typename HiddenClusters::iterator it)
- void **addVar** (std::size_t var_index)

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/EvidenceManager.h

8.49 EFG::factor::SimplyAntiCorrelatedDomainGen< N > Class Template Reference

Similar to Factor(const categoric::Group &, const UseSimpleCorrelation &), but considering a simple anti-correlation. Therefore, to all combinations having all equal values, an image equal to 0 is assigned. All the other ones, are assigned a value equal to 1. For instance assume to pass a variable set equal to: {<A: size 2>, <B: size 2>}. Then, the following combinations map is built: <0,0,0> -> 0 <0,0,1> -> 1 <0,0,2> -> 1.

```
#include <SimpleCorrelations.h>
```

Public Member Functions

- **SimplyAntiCorrelatedDomainGen** (categoric::VarStateSize var_size)
- std::optional< [misc::Chunk](#) > **next** ()

8.49.1 Detailed Description

```
template<std::size_t N>
class EFG::factor::SimplyAntiCorrelatedDomainGen< N >
```

Similar to Factor(const categoric::Group &, const UseSimpleCorrelation &), but considering a simple anti-correlation. Therefore, to all combinations having all equal values, an image equal to 0 is assigned. All the other ones, are assigned a value equal to 1. For instance assume to pass a variable set equal to: {<A: size 2>, <B: size 2>}. Then, the following combinations map is built: <0,0,0> -> 0 <0,0,1> -> 1 <0,0,2> -> 1.

```
<0,1,0> -> 1 <0,1,1> -> 1 <0,1,2> -> 1
```

```
<0,2,0> -> 1 <0,2,1> -> 1 <0,2,2> -> 1
```

```

<1,0,0> -> 1 <1,0,1> -> 1 <1,0,2> -> 1
<1,1,0> -> 1 <1,1,1> -> 0 <1,1,2> -> 1
<1,2,0> -> 1 <1,2,1> -> 1 <1,2,2> -> 1
<2,0,0> -> 1 <2,0,1> -> 1 <2,0,2> -> 1
<2,1,0> -> 1 <2,1,1> -> 1 <2,1,2> -> 1
<2,2,0> -> 1 <2,2,1> -> 1 <2,2,2> -> 0

```

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/factor/SimpleCorrelations.h

8.50 EFG::factor::SimplyCorrelatedDomainGen< N > Class Template Reference

Describe a simple correlating domain. All the variables in the passed group should have the same size. The images pertaining to the combinations having all values equal, are assumed equal to 1, all the others to 0. For instance assume to pass a variable set equal to: {<A: size 3>, <B: size 3>, <C: size 3>}. Then, the following combinations map is built: <0,0,0> -> 1 <0,0,1> -> 0 <0,0,2> -> 0.

```
#include <SimpleCorrelations.h>
```

Public Member Functions

- **SimplyCorrelatedDomainGen** (categoric::VarStateSize var_size)
- std::optional< std::pair< categoric::Combination< N >, float > > **next** ()

8.50.1 Detailed Description

```

template<std::size_t N>
class EFG::factor::SimplyCorrelatedDomainGen< N >

```

Describe a simple correlating domain. All the variables in the passed group should have the same size. The images pertaining to the combinations having all values equal, are assumed equal to 1, all the others to 0. For instance assume to pass a variable set equal to: {<A: size 3>, <B: size 3>, <C: size 3>}. Then, the following combinations map is built: <0,0,0> -> 1 <0,0,1> -> 0 <0,0,2> -> 0.

```

<0,1,0> -> 0 <0,1,1> -> 0 <0,1,2> -> 0
<0,2,0> -> 0 <0,2,1> -> 0 <0,2,2> -> 0
<1,0,0> -> 0 <1,0,1> -> 0 <1,0,2> -> 0
<1,1,0> -> 0 <1,1,1> -> 1 <1,1,2> -> 0
<1,2,0> -> 0 <1,2,1> -> 0 <1,2,2> -> 0
<2,0,0> -> 0 <2,0,1> -> 0 <2,0,2> -> 0
<2,1,0> -> 0 <2,1,1> -> 0 <2,1,2> -> 0
<2,2,0> -> 0 <2,2,1> -> 0 <2,2,2> -> 1

```

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/factor/SimpleCorrelations.h

8.51 EFG::misc::Slot< T > Struct Template Reference

Public Member Functions

- **Slot** (const [Slot](#) &)=delete
- [Slot](#) & **operator=** (const [Slot](#) &)=delete
- **Slot** ([Slot](#) &&) noexcept
- [Slot](#) & **operator=** ([Slot](#) &&) noexcept
- void **transferIntoPool** ([misc::MemoryPool](#) &pool)
- std::size_t **getMemoryFootprint** () const
- auto **get** () const

Static Public Member Functions

- static [Slot](#) **makeNonOwning** (std::span< T > the_view)
- static [Slot](#) **makeOwning** (std::span< const T > the_view)

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/MemoryPool.h

8.52 EFG::structure::Structure Struct Reference

Classes

- struct [BinaryFactorInStructure](#)
data

Public Member Functions

- **Structure** ([ModelSeed](#) seed)
- **Structure** (const [Structure](#) &)=delete
- [Structure](#) & **operator=** (const [Structure](#) &)=delete
- **Structure** ([Structure](#) &&) noexcept=delete
- [Structure](#) & **operator=** ([Structure](#) &&) noexcept=delete
- template<typename VarsGen >
requires misc::IsGenerator< VarsGen, std::size_t > void **updateClusters** (VarsGen vars)
modifiers
- template<typename M >
void **registerManager** (M &manager)
- void **addListener** ([WorkerPoolActivityListener](#) &listener)
- std::vector< float > **make_messages_pool_swap** () const
- void **swap_messages_pool** (std::vector< float > &swap)
- auto **get_active_incoming** (std::size_t var_index) const
getters
- auto **get_message_dependencies_rng** (const [MessageMetaData](#) &msg) const
- template<typename M >
M * **getManager** ()
- auto & **getMessageData** (MessageMetaData::MsgIdx idx)
- std::span< float > **getMessageValues** (const [MessageMetaData](#) &msg, std::vector< float > &buffer)
- std::span< float > **getMessageValues** (const [MessageMetaData](#) &msg)
- std::span< const float > **getMessageValues** (const [MessageMetaData](#) &msg) const
- auto & **getListeners** ()
- std::size_t **locateBinaryFactor** (std::size_t var_a, std::size_t var_b) const

Public Attributes

- [Config](#) **config**
- NamedVariables **named_vars_table**
- `std::vector< std::size_t >` **permanent_evidences**
- `std::vector< Node >` **nodes**
- `std::vector< BinaryFactorInStructure >` **binary_factors**
- [Tunability](#) **tunability**
- HiddenClusters **clusters**

The documentation for this struct was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/Structure.h`

8.53 EFG::structure::Trainer Struct Reference

Classes

- struct [NoCB](#)

Public Member Functions

- [Trainer](#) & **max_iterations** (`std::size_t` val)
- [Trainer](#) & **gradient_rescale** (`float` val)
- [Trainer](#) & **advance_toll_percentage** (`float` val)
- `template<typename CB >`
`std::size_t` **train_model_with_cb** ([TunableWeightsManager](#) &model, `std::shared_ptr< const misc::Samples >`
`training_set`, `CB cb`) `const`
- `std::size_t` **train_model** ([TunableWeightsManager](#) &model, `std::shared_ptr< const misc::Samples >`
`training_set`) `const`

8.53.1 Member Function Documentation

8.53.1.1 train_model_with_cb()

```
template<typename CB >
std::size_t EFG::structure::Trainer::train_model_with_cb (
    TunableWeightsManager & model,
    std::shared_ptr< const misc::Samples > training_set,
    CB cb ) const [inline]
```

Returns

the number of actually performed iterations

The documentation for this struct was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/TunableWeightsManager.h`

8.54 EFG::structure::Tunability Struct Reference

Classes

- struct [TunableBinaryFactor](#)
- struct [TunableUnaryFactor](#)

Public Member Functions

- template<typename T >
auto **makeFilteredRange** () const

Public Attributes

- std::vector< std::variant< [TunableUnaryFactor](#), [TunableBinaryFactor](#) > > **order**

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/ModelSeed.h

8.55 EFG::structure::Tunability::TunableBinaryFactor Struct Reference

Public Attributes

- std::size_t **first_var_index**
- std::size_t **second_var_index**

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/ModelSeed.h

8.56 EFG::structure::Tunability::TunableUnaryFactor Struct Reference

Public Attributes

- std::size_t **var_index**

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/ModelSeed.h

8.57 EFG::structure::TunableWeightsManager::TunableWeightsGradient Struct Reference

Public Member Functions

- **TunableWeightsGradient** ([TunableWeightsManager](#) &source, std::shared_ptr< const [misc::Samples](#) > training_set)
- void **get** (std::vector< float > &recipient)

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/TunableWeightsManager.h

8.58 EFG::structure::TunableWeightsManager Class Reference

Classes

- struct [TunableWeightsGradient](#)

Public Member Functions

- [TunableWeightsGradient](#) **gradient** (std::shared_ptr< const [misc::Samples](#) > training_set)
- void **setTunableWeights** (const std::vector< float > &values)
- void **setTunableWeights** (std::span< const float > values)
- void **getTunableWeights** (std::vector< float > &recipient) const

Protected Member Functions

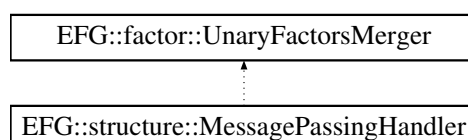
- void **init** (StructurePtr context)

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/TunableWeightsManager.h

8.59 EFG::factor::UnaryFactorsMerger Struct Reference

Inheritance diagram for EFG::factor::UnaryFactorsMerger:



Public Member Functions

- `std::span< float > getMerged ()`
- `void reset (categoric::VarStateSize var_size)`
- `template<bool Normalize>`
`void merge (std::span< const float > to_merge)`
- `template<bool Normalize, typename Transform >`
`void mergeFactor (const factor::FactorT< 1, Transform > &to_merge)`
- `void normalize ()`

The documentation for this struct was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/factor/Factor.h`

8.60 EFG::misc::UniformSampler Class Reference

Public Member Functions

- `std::size_t sampleFromDiscrete (const std::vector< float > &distribution) const`
- `std::size_t sampleFromDiscrete (std::span< const float > distribution) const`
- `void resetSeed (std::size_t newSeed)`

The documentation for this class was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/UniformSampler.h`

8.61 EFG::misc::ValuesPool Class Reference

Public Member Functions

- `std::span< float > get_slot (std::size_t slot_idx, std::span< float > giver)`
- `std::span< const float > get_slot (std::size_t slot_idx, std::span< const float > giver) const`

Static Public Member Functions

- `template<typename SlotsSizesRng >`
`static std::pair< ValuesPool, std::size_t > make (SlotsSizesRng rng)`

The documentation for this class was generated from the following file:

- `/home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/MemoryPool.h`

8.62 EFG::structure::ModelBuilder::VariableDescription Struct Reference

Public Attributes

- bool **isPermanentEvidence** {false}
- std::string **label** {""}

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/ModelSeed.h

8.63 EFG::misc::VectorCache< Ts > Class Template Reference

Public Member Functions

- template<std::size_t Index, bool Clear = true>
auto & **getBuffer** ()

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/VectorCache.h

8.64 EFG::misc::VectorCache< T > Class Template Reference

Public Member Functions

- template<bool Clear = true>
auto & **getBuffer** ()

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/VectorCache.h

8.65 EFG::misc::WorkerPool Class Reference

Public Member Functions

- **WorkerPool** (std::size_t pool_size)
- **WorkerPool** (const [WorkerPool](#) &)=delete
- **WorkerPool** & **operator=** (const [WorkerPool](#) &)=delete
- template<typename Pred >
void **compute** (Pred pred)
- std::size_t **size** () const noexcept

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/misc/WorkerPool.h

8.66 EFG::structure::WorkerPoolActivityListener Struct Reference

Public Member Functions

- virtual void **updatePool** ([misc::WorkerPool](#) *pool)
- auto * **getPool** ()

The documentation for this struct was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/Structure.h

8.67 EFG::structure::WorkerPoolManager Class Reference

Classes

- struct [ScopedActivator](#)

Public Member Functions

- std::unique_ptr< [ScopedActivator](#) > **activatePool** ()

Protected Member Functions

- void **init** (StructurePtr context)

The documentation for this class was generated from the following file:

- /home/andrea/Scrivania/GitProj/Easy-Factor-Graph/src/header/EasyFactorGraph/structure/WorkerPool↔
Manager.h

Index

`delta_iterations`
 EFG::structure::GibbsSampler::SamplesGenerationContext, 77

EFG, 47

EFG::categoric, 47

EFG::categoric::GroupT< Support >, 67

EFG::categoric::RangeT< Support >, 76

 RangeT, 76

EFG::Error, 59

EFG::factor, 48

 get_probabilities, 49

EFG::factor::ExponentialTrasform, 62

 getWeight, 62

EFG::factor::FactorT< 1, Transform >, 64

EFG::factor::FactorT< N, Transform >, 63

EFG::factor::NullTrasform, 73

EFG::factor::SimplyAntiCorrelatedDomainGen< N >, 79

EFG::factor::SimplyCorrelatedDomainGen< N >, 80

EFG::factor::UnaryFactorsMerger, 84

EFG::misc, 50

 IsGenerator, 51

EFG::misc::Chunk, 56

EFG::misc::ConcurrentSafeLevels< T >, 56

EFG::misc::GeneratorMap< U, T, GenT >, 64

EFG::misc::GenFromBlock< T >, 65

EFG::misc::Intervals, 67

EFG::misc::Intervals::IntervalsIter, 68

EFG::misc::MemoryPool, 68

EFG::misc::ModeToStream< FileMode::READ >, 72

EFG::misc::ModeToStream< FileMode::WRITE >, 72

EFG::misc::ModeToStream< M >, 71

EFG::misc::Range, 75

EFG::misc::Samples, 77

EFG::misc::Samples::SamplesIter, 78

EFG::misc::Slot< T >, 81

EFG::misc::UniformSampler, 85

EFG::misc::ValuesPool, 85

EFG::misc::VectorCache< T >, 86

EFG::misc::VectorCache< Ts >, 86

EFG::misc::WorkerPool, 86

EFG::model, 51

EFG::model::ConditionalRandomField, 56

 makeSamplesSpanningEvidences, 57

EFG::model::Graph, 66

EFG::model::Model< Components >, 70

EFG::structure, 52

 load_train_set, 53

EFG::structure::BeliefManager, 55

EFG::structure::Config, 57

EFG::structure::ConfigManager, 58

 setCalibrationTolerance, 58

 setMaxCalibrationIter, 58

 setWorkersPoolSize, 58

EFG::structure::Evidence, 60

 NOT_AN_EVIDENCE, 60

EFG::structure::EvidenceRemoveManager, 60

 removeEvidences, 61

EFG::structure::EvidenceSetManager, 61

 setEvidences, 62

EFG::structure::GibbsSampler, 65

 makeSamples, 65

EFG::structure::GibbsSampler::SamplesGenerationContext, 77

 delta_iterations, 77

 transient, 77

EFG::structure::HiddenCluster, 67

EFG::structure::MessageMetaData, 69

EFG::structure::MessageMetaData::FactorInfo, 63

EFG::structure::MessagePassingHandler, 69

EFG::structure::ModelBuilder, 70

EFG::structure::ModelBuilder::VariableDescription, 86

EFG::structure::ModelExport, 71

EFG::structure::ModelSeed, 71

EFG::structure::Node, 72

EFG::structure::PairHasher, 73

EFG::structure::PropagationOrder, 73

EFG::structure::PropagationResult, 73

EFG::structure::QueryManager, 74

 getHiddenSetMAP, 74

 getMAP, 74

 getMarginalDistribution, 75

EFG::structure::ScopedEvidenceUpdate, 79

EFG::structure::ScopedEvidenceUpdate::Context, 59

EFG::structure::Structure, 81

EFG::structure::Structure::BinaryFactorInStructure, 55

EFG::structure::Trainer, 82

 train_model_with_cb, 82

EFG::structure::Trainer::NoCB, 72

EFG::structure::Tunability, 83

EFG::structure::Tunability::TunableBinaryFactor, 83

EFG::structure::Tunability::TunableUnaryFactor, 83

EFG::structure::TunableWeightsManager, 84

EFG::structure::TunableWeightsManager::TunableWeightsGradient, 84

EFG::structure::WorkerPoolActivityListener, 87

EFG::structure::WorkerPoolManager, 87

EFG::structure::WorkerPoolManager::ScopedActivator,

78

get_probabilities
 EFG::factor, 49
getHiddenSetMAP
 EFG::structure::QueryManager, 74
getMAP
 EFG::structure::QueryManager, 74
getMarginalDistribution
 EFG::structure::QueryManager, 75
getWeight
 EFG::factor::ExponentialTrasform, 62

IsGenerator
 EFG::misc, 51

load_train_set
 EFG::structure, 53

makeSamples
 EFG::structure::GibbsSampler, 65
makeSamplesSpanningEvidences
 EFG::model::ConditionalRandomField, 57

NOT_AN_EVIDENCE
 EFG::structure::Evidence, 60

RangeT
 EFG::categoric::RangeT < Support >, 76
removeEvidences
 EFG::structure::EvidenceRemoveManager, 61

setCalibrationTolerance
 EFG::structure::ConfigManager, 58
setEvidences
 EFG::structure::EvidenceSetManager, 62
setMaxCalibrationIter
 EFG::structure::ConfigManager, 58
setWorkersPoolSize
 EFG::structure::ConfigManager, 58

train_model_with_cb
 EFG::structure::Trainer, 82
transient
 EFG::structure::GibbsSampler::SamplesGenerationContext,
 77